RICHFIELD GRADUATE INSTITUTE OF TECHNOLOGY

Faculty of Information Technology

Bachelor of Science in Information Technology (BSc IT) | IT Project (Year 3), Semester 2 2026

HireTrack

Applicant Tracking & Resource Allocation System

Phase 5: Implementation (Sections 5.1 to 5.5)

Item	Detail
Project title	HireTrack: Applicant Tracking & Resource Allocation System
Student number	402311822
Institution	Richfield Graduate Institute of Technology
Qualification	BSc IT, IT Project (Year 3)
Supervisor	Rhumani
Date	11 November 2026
Source code	https://github.com/RohanRajaramNaik/hiretrack

Table of Contents

5.1 Introduction 4
5.1.1 Purpose of the Implementation Phase 4
5.1.2 Scope of this release 4
5.1.3 Technology stack and reasons 4
5.2 Coding 6
5.2.1 System architecture 6
5.2.2 Module breakdown 7
5.2.3 Database schema 7
5.2.4 Key code excerpts 9
5.2.5 Coding standards, validation and error handling 12
5.2.6 Security measures 12
5.2.7 User interface 12
5.3 Testing 24
5.3.1 Approach 24
5.3.2 Unit and integration results 24
5.3.3 Coverage 25
5.3.4 Performance 26
5.3.5 Debugging and code improvement 26
5.4 System Testing 27
5.4.1 Approach and requirements 27
5.4.2 Test cases 29
5.4.3 Evaluation of the testing results 33
5.4.4 Defects found and how they were fixed 33
5.4.5 Limitations 34
5.4.6 Recommendations 34
5.5 Installation 35
5.5.1 Summary 35
5.5.2 Steps 35
5.5.3 Verification on a clean copy 35
Conclusion 36
References 36
Appendix A: Core Backend Source Code 37
backend/app/main.py 37
backend/app/models.py 38
backend/app/security.py 42
backend/app/engine/scoring.py 43
backend/app/engine/skills.py 47
backend/app/engine/embeddings.py 49
backend/app/engine/pipeline.py 50

backend/app/engine/allocation.py 52
backend/app/services/match_service.py 52
backend/app/services/pipeline_service.py 55
backend/app/services/resource_service.py 58

5.1 Introduction

5.1.1 Purpose of the Implementation Phase

The Implementation Phase turns the System Design (Phase 4, 30 October 2026) into working, tested and installable software. It answers three questions: does the system do what the requirements say, does it do it reliably and fast enough, and can someone else install and run it from the repository alone.

This document covers the five parts the Faculty guidelines require for this phase: coding (5.2), testing (5.3), system testing with test cases and an evaluation of the results (5.4), and installation (5.5).

5.1.2 Scope of this release

The Design phase narrowed the first release to the candidate-job matching engine and listed user accounts, the applicant pipeline and the recruiter dashboard as future scope. The Proposal (28 August 2026), however, described HireTrack as a full applicant tracking and resource allocation system. Because the matching engine was finished early, the group brought the Proposal's remaining objectives into this release. The matching engine was kept exactly as designed and became one module of the larger system.

Area	Status in Design (Phase 4)	Status in this release
Candidate and job management, four-part scoring, ranking, CSV export, backups	In scope	Built as designed; job location and open/closed status added
User accounts and roles (JWT + bcrypt)	Future scope	Built: Administrator, Recruiter (HR), Resource Manager
Applicant pipeline with audit history	Future scope	Built: Applied, Screened, Interview, Offer, Hired, Rejected
Recruiter dashboard	Future scope	Built: key counts and five charts
Uploading CV files	Future scope	Built: PDF, Word and text CVs pre-fill the form
Bulk import	Not planned	Built: CSV and Excel with a per-row error report
Resource allocation (staff, projects, utilisation)	Proposal title only	Built: allocations by percentage and date with no over-booking
Multi-user access over a network	Future scope	Still future scope (runs on 127.0.0.1)

Table 5.1: Scope of the release compared with the Design phase

The requirements were extended to match. FR1 to FR13 and NFR1 to NFR8 from the Analysis phase still apply. FR14 to FR26 and NFR9 to NFR10 were added for the new modules and are listed in Section 5.4.1.

5.1.3 Technology stack and reasons

The stack is the one chosen in the Planning and Design phases. No paid service or cloud platform is used.

Layer	Technology	Why it was chosen
Frontend	React 18, Vite 5, Tailwind CSS 3	Component model suits many similar screens; Vite builds fast; Tailwind keeps one consistent visual style without a heavy component library.
Backend	Python 3.11, FastAPI, Pydantic 2, Uvicorn	The matching engine and the model library are Python. FastAPI validates every request from type hints and generates API documentation at /docs.
Data access	SQLAlchemy 2 ORM	Bound parameters only (no SQL injection), and the same models define the tables and their constraints.
Database	SQLite	One file, no server to install, supports foreign keys and CHECK constraints, easy to back up.
Matching	sentence-transformers all-MiniLM-L6-v2	Small (about 90 MB), runs on an ordinary CPU, works offline, free. Finds candidates who describe the work in different words.
Security	bcrypt (passlib), PyJWT	Industry-standard password hashing and signed, expiring sessions, as planned in the Proposal.
Import	openpyxl, pypdf	Read Excel files and PDF CVs without Microsoft Office.
Testing	pytest, pytest-cov, Vitest, React Testing Library, Playwright	One tool per level: unit and API (pytest), screens (Vitest), full journeys in a real browser (Playwright).

Table 5.2: Technology stack

The system follows the linear life cycle with iterative build cycles chosen in Planning: each module was built, unit tested and reviewed before the next one started, and the full suites were re-run after every change.

5.2 Coding

The complete source code is published in the GitHub repository https://github.com/RohanRajaramNaik/hiretrack. It contains the backend, the frontend, the seed data, the tests, the start scripts, README.md and INSTALLATION_GUIDE.md. The key excerpts below are explained in this section, and the full listing of the core backend files is in Appendix A.

5.2.1 System architecture

HireTrack is a three-tier client-server system that runs on one computer (Figure 5.1). The browser shows a React single-page application. It talks to a FastAPI server over HTTP using JSON. The server keeps everything in a SQLite file. In normal use the server also serves the built screens, so the whole system uses one port.

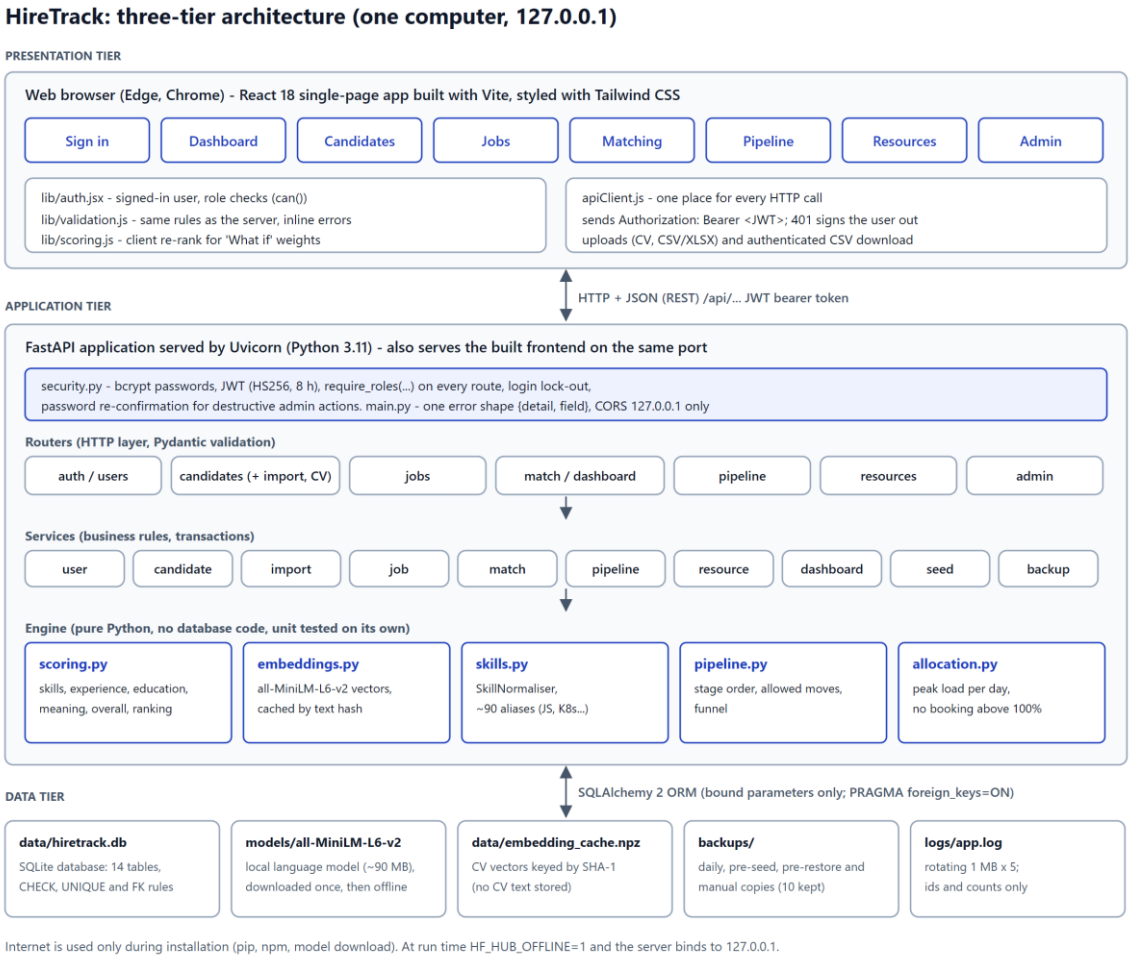


Figure 5.1: Three-tier architecture of HireTrack

Inside the server the code is layered. Routers handle HTTP and validation, services hold the business rules and the database transactions, and the engine modules hold pure rules with no database code. This means the rules that matter most (scoring, stage changes, allocation limits) can be tested on their own in milliseconds.

5.2.2 Module breakdown

Module	Main files	Responsibility	Requirements
M1 Accounts and security	security.py, routers/auth.py, services/user_service.py, pages/Login.jsx, lib/auth.jsx	Sign-in, sessions, role checks, user administration, password change and lock-out	FR14, FR15, NFR9
M2 Candidate management	services/candidate_service.py, services/import_service.py, pages/Candidates.jsx, ImportPanel.jsx	Candidate CRUD and search; CV text extraction; CSV/Excel import	FR1, FR2, FR12, FR16, FR17
M3 Job specifications	services/job_service.py, pages/Jobs.jsx	Job CRUD with skills, location, status and weights	FR3, FR4
M4 Matching engine	engine/scoring.py, engine/embeddings.py, engine/skills.py, services/match_service.py, pages/Results.jsx	Four scores, overall score, ranking, explanations, what-if re-ranking, CSV export	FR5 to FR10, FR13, NFR1, NFR7
M5 Applicant pipeline	engine/pipeline.py, services/pipeline_service.py, pages/Pipeline.jsx	Applications, stage rules, audit history, shortlist from results, hire into staff pool	FR18 to FR21
M6 Resource allocation	engine/allocation.py, services/resource_service.py, pages/Resources.jsx	Staff, projects, allocations, no booking above 100%, utilisation and bench	FR22 to FR24
M7 Dashboard	services/dashboard_service.py, pages/Dashboard.jsx, components/Charts.jsx	Key counts, funnel, applicants per job, utilisation, hires per month, activity	FR25
M8 Administration	services/seed_service.py, services/backup_service.py, routers/admin.py, pages/Admin.jsx, seed/	Seed datasets, backups, integrity-checked restore	FR11, FR26

Table 5.3: Modules, files and the requirements each one covers

5.2.3 Database schema

The database extends the Phase 4 design (Figure 5.2). The seven tables from the design are unchanged apart from two columns on the job specification (location and status) and two optional contact columns on the candidate. Seven tables were added for accounts, the pipeline and resource allocation. The design is in Third Normal Form: each non-key column depends only on its table's key, lists are stored in link tables, and lookups (education level, skill) are separate tables.

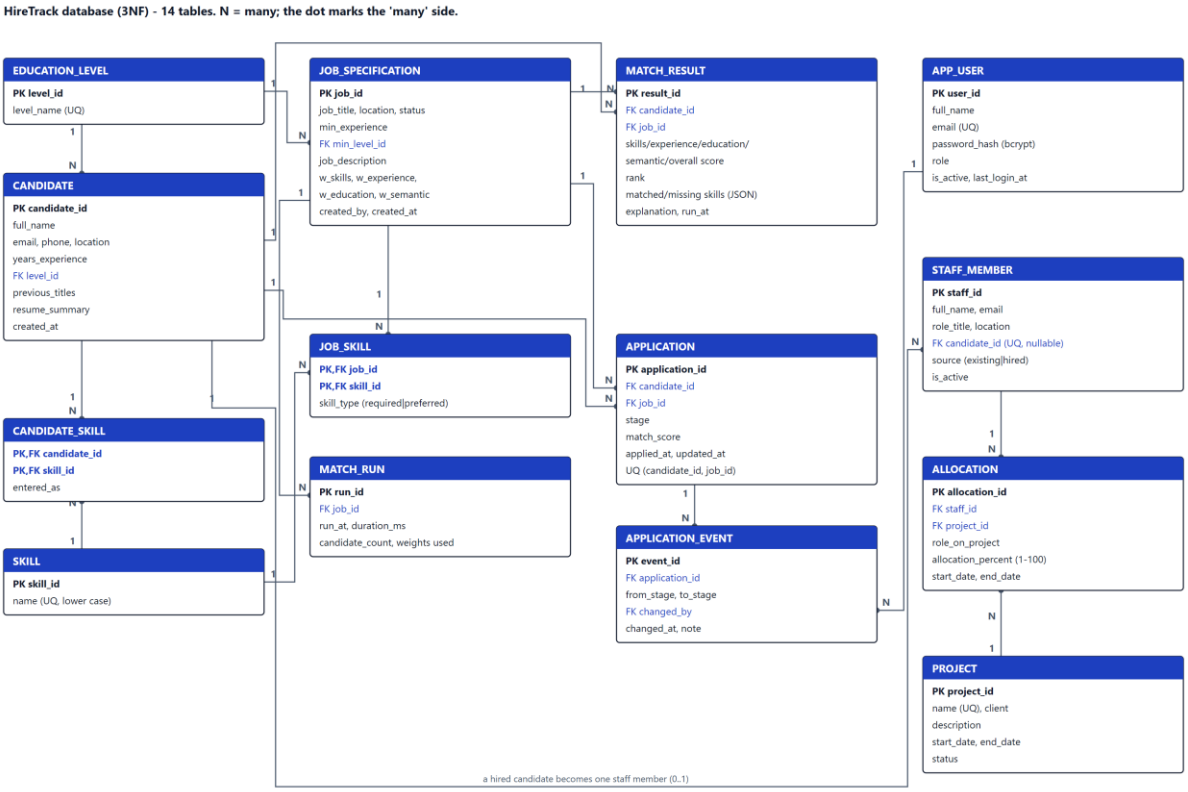


Figure 5.2: Entity-relationship diagram (14 tables)

Table	Purpose	Key rules (enforced by the database)	Rows in seed
app_user	Sign-in accounts	email UNIQUE and lower case; role IN (admin, recruiter, resource_manager)	3
candidate	Talent pool	name 2-100 characters; years 0-50; summary 20-3000 characters	600
job_specification	Roles to fill	location 2-80 characters; status IN (open, closed); weights 0-100 and total = 100	33
application	Candidate applying for a job	stage IN (6 stages); UNIQUE (candidate_id, job_id); cascades with candidate and job	847
application_event	Audit history of stage changes	one row per change; changed_by kept as SET NULL if a user is removed	1906
staff_member	People who can be allocated	candidate_id UNIQUE (a hire becomes one staff member); source IN (existing, hired)	73
project	Client projects	name UNIQUE; end_date >= start_date; status IN (planned, active, on_hold, completed)	12
allocation	Share of a person's time on a project	allocation_percent 1-100; end_date >= start_date; the 100% daily limit is enforced by the service	102

Table 5.4: New and changed tables (the other six are as in the Design phase)

An existing database from the Phase 4 build is upgraded automatically when the server starts: missing columns are added with safe defaults, missing tables are created and the default accounts are added if there are none. Restoring an old backup goes through the same step.

5.2.4 Key code excerpts

The excerpts below are copied from the source files. Each shows a rule the requirements depend on.

a) Skills score (FR5)

Required skills are worth 80 points and preferred skills 20. Skills are compared after normalisation, so "JS" on a CV matches "JavaScript" on a job. The function returns which skills matched and which are missing, which the results screen shows.

backend/app/engine/scoring.py

```
def skills_score(candidate_skills: Sequence[str], required: Sequence[str], preferred: Sequence[str])
-> SkillMatch:
    """Required skills are worth 80 points, preferred 20.

    With no preferred skills, required skills are worth the full 100.
    Inputs must already be canonical (see SkillNormaliser).
    """
    have = set(candidate_skills)
    matched_req = [s for s in required if s in have]
    missing_req = [s for s in required if s not in have]
    matched_pref = [s for s in preferred if s in have]
    missing_pref = [s for s in preferred if s not in have]

    if not required:
        req_part = 0.0
    elif preferred:
        req_part = len(matched_req) / len(required) * REQUIRED_SHARE
    else:
        req_part = len(matched_req) / len(required) * 100
    pref_part = len(matched_pref) / len(preferred) * PREFERRED_SHARE if preferred else 0.0

    return SkillMatch(
        score=round1(clamp(req_part + pref_part)),
        matched_required=matched_req,
        missing_required=missing_req,
        matched_preferred=matched_pref,
        missing_preferred=missing_pref,
    )
```

b) Overall score (FR9)

The overall score is a weighted average of the four scores. It is calculated in whole tenths with round-half-up, so the browser's instant "what if" re-ranking, which uses the same method in JavaScript, always agrees with the server to the last decimal.

backend/app/engine/scoring.py

```
def overall_score(skills: float, experience: float, education: float, semantic: float, weights:
Weights) -> float:
    """Weighted average of the four components, rounded to 1 decimal.

    Computed in integer tenths so the browser's "what if" re-ranking (which
    uses the same integer method) always agrees with the server exactly.
    """
    tenths = [int(round(c * 10)) for c in (skills, experience, education, semantic)]
    ws = (weights.skills, weights.experience, weights.education, weights.semantic)
    total = sum(t * w for t, w in zip(tenths, ws))  # in (tenths x percent)
    return ((total + 50) // 100) / 10  # round half up back to tenths
```

c) Pipeline stage rules (FR18)

An application moves forward one stage at a time or is rejected. Hired and Rejected are final, except that a rejection can be reopened at the stage it was rejected from. The function raises an error with a reason that the screen shows to the user.

backend/app/engine/pipeline.py

```
def check_move(current: str, target: str, reopen_to: str | None = None) -> None:
    """Raise StageError with a plain-English reason if current -> target is not allowed."""
    if target not in ALL_STAGES:
        raise StageError(f"Unknown stage '{target}'")
    if current == target:
        raise StageError(f"The application is already at {LABELS[current]}")
    if current in FINAL and not (current == REJECTED and target == reopen_to):
        raise StageError(f"{LABELS[current]} is a final stage and cannot be changed")
    if target not in allowed_moves(current, reopen_to):
        nxt = next_stage(current)
        hint = f" The next stage is {LABELS[nxt]}." if nxt else ""
        raise StageError(f"Cannot move from {LABELS[current]} to {LABELS[target]}.{hint}")
```

d) Changing a stage: audit history and hiring (FR19, FR21)

The stage change and its history row are written in the same transaction, so the history can never disagree with the current stage. A move to Hired also creates the staff record.

backend/app/services/pipeline_service.py

```
def change_stage(db: Session, application_id: int, data: StageChangeIn, user: User) -> Application:
    app = get_application(db, application_id)
    try:
        rules.check_move(app.stage, data.stage, _rejected_from(app))
    except rules.StageError as exc:
        raise AppError(str(exc), field="stage", status_code=422) from exc
    if data.stage == "hired" and app.job.status != "open":
        raise AppError("This job is closed, so nobody else can be hired for it", field="stage",
status_code=422)
    now = utcnow()
    app.events.append(
        ApplicationEvent(
            from_stage=app.stage, to_stage=data.stage, changed_by=user.user_id, changed_at=now,
note=data.note or None
        )
    )
    app.stage = data.stage
    app.updated_at = now
    if data.stage == "hired":
        _ensure_staff(db, app)
    db.commit()
    log.info("Application id=%s moved to %s by user id=%s", application_id, data.stage,
user.user_id)
    db.expire_all()
    return get_application(db, application_id)
```

e) Preventing over-allocation (FR23)

A person's load can only change on the day a booking starts or the day after one ends. The check therefore looks only at those days instead of every day in the range, and reports the first day the limit would be broken.

backend/app/engine/allocation.py

```python
def peak_load(bookings: Iterable[Booking], start: date, end: date) -> tuple[int, date | None]:
    """Highest total load on any day in [start, end], and the first day it happens.

    Load only changes on a booking's start day or the day after its end, so it
    is enough to check those days (a sweep), not every day in the range.
    """
    inside = [b for b in bookings if overlaps(b.start, b.end, start, end)]
    if not inside:
        return 0, None
    days = {start}
    for b in inside:
        if start <= b.start <= end:
            days.add(b.start)
        after = b.end + timedelta(days=1)
        if start <= after <= end:
            days.add(after)
    best, best_day = 0, None
    for d in sorted(days):
        load = load_on(inside, d)
        if load > best:
            best, best_day = load, d
    return best, best_day
```

f) Role-based access on the server (FR14, NFR9)

Every route declares the roles it allows. The check runs on the server, so hiding a button in the browser is never the only protection.

backend/app/security.py

```python
def require_roles(*roles: str):
    """Dependency factory: allow only users whose role is in `roles` (else 403)."""

    def _check(user=Depends(current_user)):
        if user.role not in roles:
            log.warning("Denied user id=%s role=%s (needs %s)", user.user_id, user.role,
"/".join(roles))
            raise Forbidden("Your role does not have access to this action.")
        return user

    return _check
```

backend/app/routers/pipeline.py

```python
@router.post("/{application_id}/stage", response_model=ApplicationOut)
```

g) Bulk import validation (FR17)

Each row of an imported file is checked with exactly the same Pydantic model as the Add Candidate form. Bad rows are reported with their row number; good rows are saved together in one transaction.

backend/app/services/import_service.py (import_candidates)

```python
for i, row in enumerate(rows, start=2):  # row 1 is the header
    try:
        data = CandidateIn(**_to_input(row))
    except ValidationError as exc:
        field, msg = _friendly(exc.errors()[0])
        errors.append(ImportRowError(row=i, field=field, message=msg))
        continue
    if data.email and (data.email in existing_emails or data.email in seen_in_file):
        skipped += 1
        continue
    if data.email:
        seen_in_file.add(data.email)
    valid.append(data)
```

5.2.5 Coding standards, validation and error handling

- **Naming and layout.** Python follows PEP 8 with type hints; JavaScript uses one component per file. Files are named after what they hold (pipeline_service.py, Pipeline.jsx).
- **Comments.** Every module starts with a docstring explaining its job and its rules. Comments explain why, not what (for example why scores are calculated in tenths).
- **Three layers of validation (NFR6).** The screen checks input as it is typed (lib/validation.js); Pydantic checks every request on the server; database CHECK, UNIQUE and FOREIGN KEY rules are the last line of defence.
- **One error shape.** Every API error is {"detail": "...", "field": "..."}. The screens use the field name to show the message under the right input.
- **Transactions.** Matching, import, seeding and stage changes write everything or nothing; any error rolls back.
- **Parameter passing.** Routers receive validated schema objects and pass them to service functions together with the database session and the signed-in user, so services never read HTTP data directly.
- **Efficiency.** CV embeddings are calculated once and cached in memory and on disk; candidates are loaded with selectinload to avoid one query per row; the candidate table is virtualised so 2,600 rows scroll smoothly.

5.2.6 Security measures

Threat	Control
Stolen passwords	Only bcrypt hashes are stored (12 rounds). Passwords need 8+ characters with a letter and a number.
Guessing passwords	Five failed sign-ins lock that email for 60 seconds. The same message is shown for a wrong email or password.
Using the system without signing in	Every API route except /api/health needs a signed JWT (HS256, 8-hour expiry). The secret is generated on first run and kept in data/.jwt_secret.
Doing more than the role allows	require_roles(...) on every route (36 role and endpoint combinations tested). Screens hide what a role cannot use.
Accidental data loss	Loading a dataset or restoring a backup needs the administrator's password again. A safety backup is taken first; restore runs PRAGMA integrity_check.
SQL injection / XSS	SQLAlchemy bound parameters only; React escapes all text; no dangerouslySetInnerHTML.
Spreadsheet formula injection	CSV export prefixes cells that start with =, +, - or @.
Network exposure	The server binds to 127.0.0.1; CORS allows only the local addresses.
Privacy (POPIA)	Only fictitious data; logs hold ids and counts, never CV text; scoring never reads name, email, age or gender.

Table 5.5: Security controls

5.2.7 User interface

The screens use one neutral palette with a single blue accent, green only for Run Match and red only for destructive actions. Every page uses the same header, table and side-panel patterns. The menu shows only what the signed-in role can use in the order a user meets them, from sign-in to administration (Figures 5.3 to 5.20).

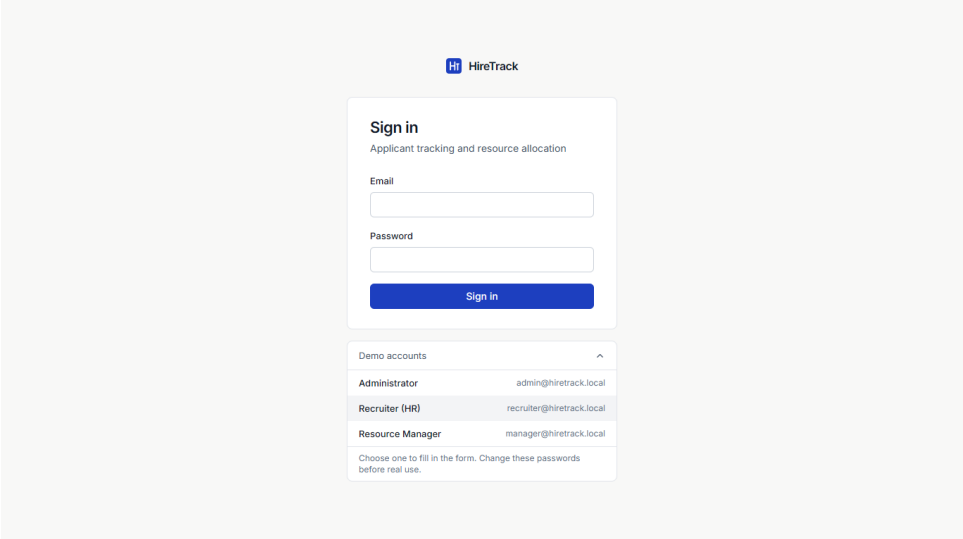


Figure 5.3: Sign-in screen with the demo accounts expanded

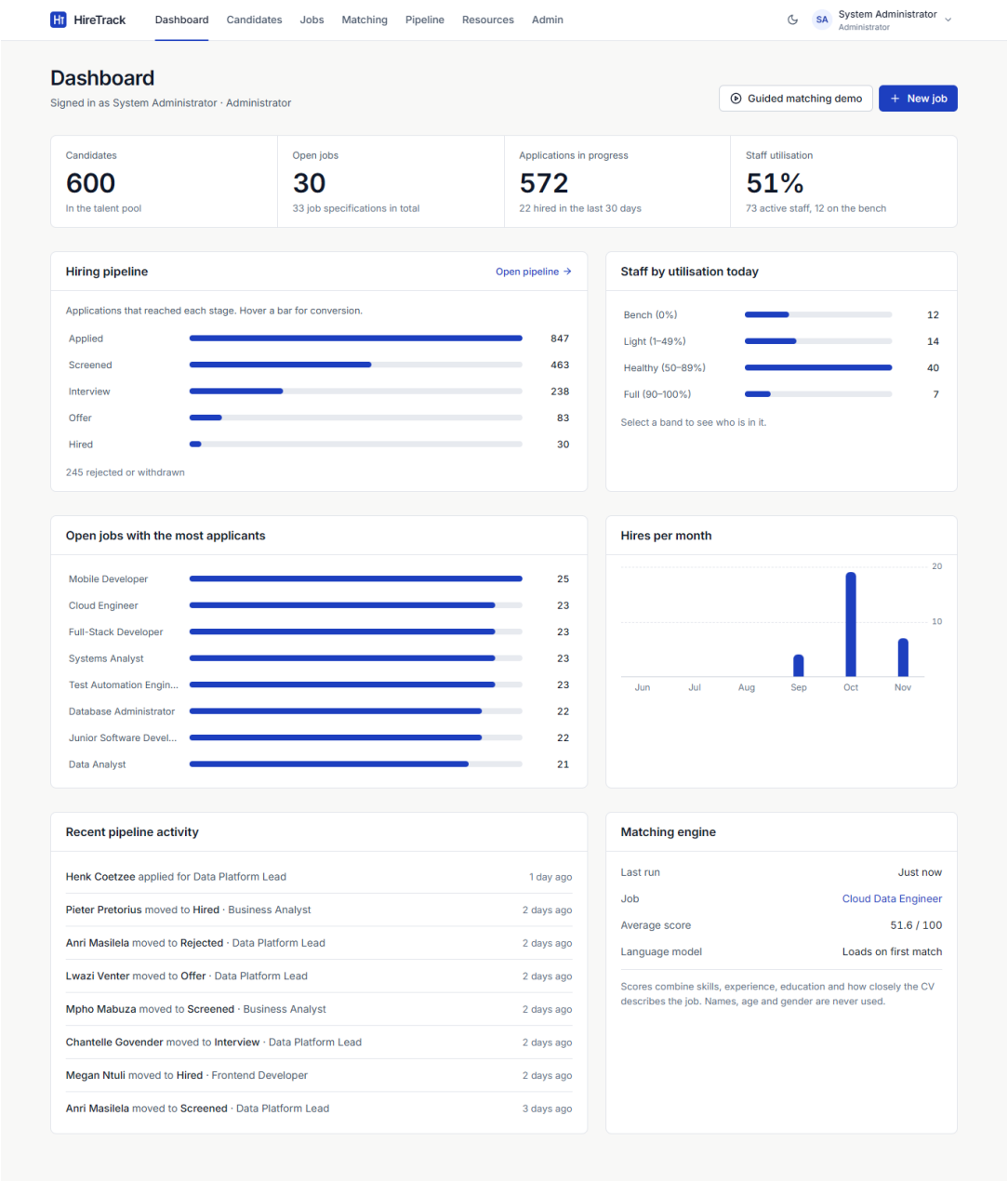


Figure 5.4: Dashboard (Administrator): key counts, pipeline funnel, utilisation, applicants per job, hires per month

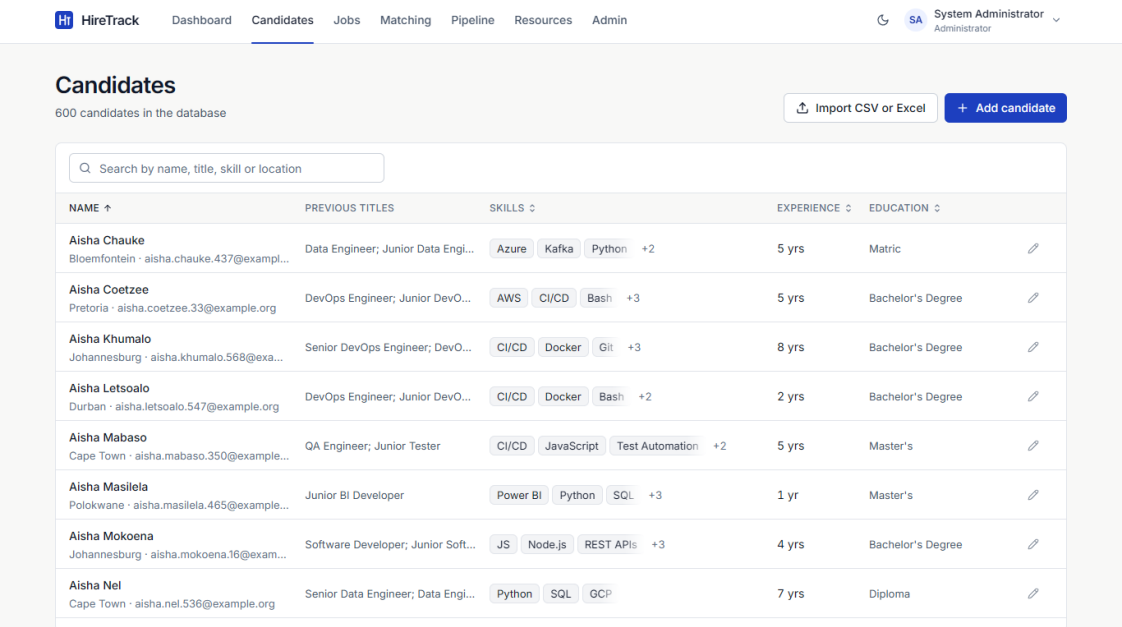


Figure 5.5: Candidate list with search and filters (600 seeded candidates)

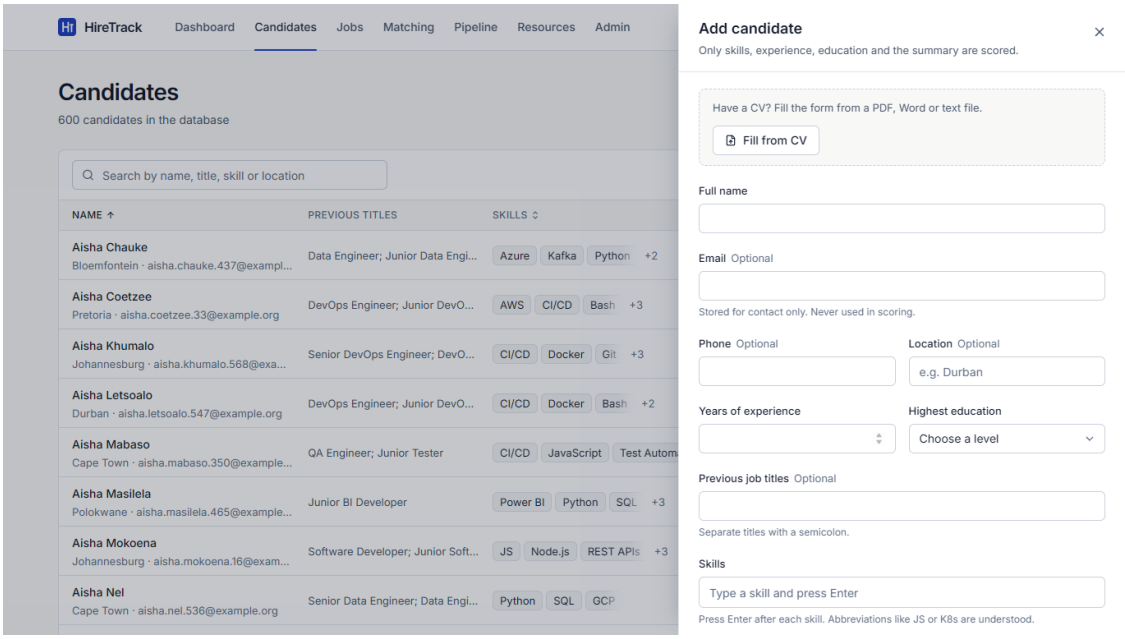


Figure 5.6: Add Candidate panel with "Fill from CV"

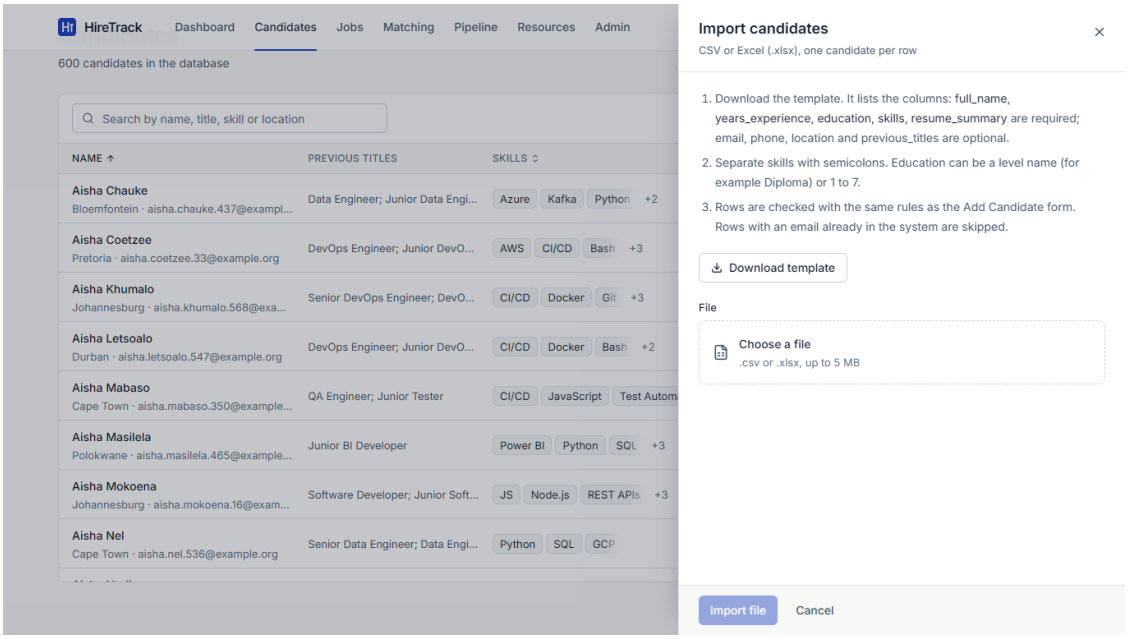


Figure 5.7: Bulk import of candidates from CSV or Excel

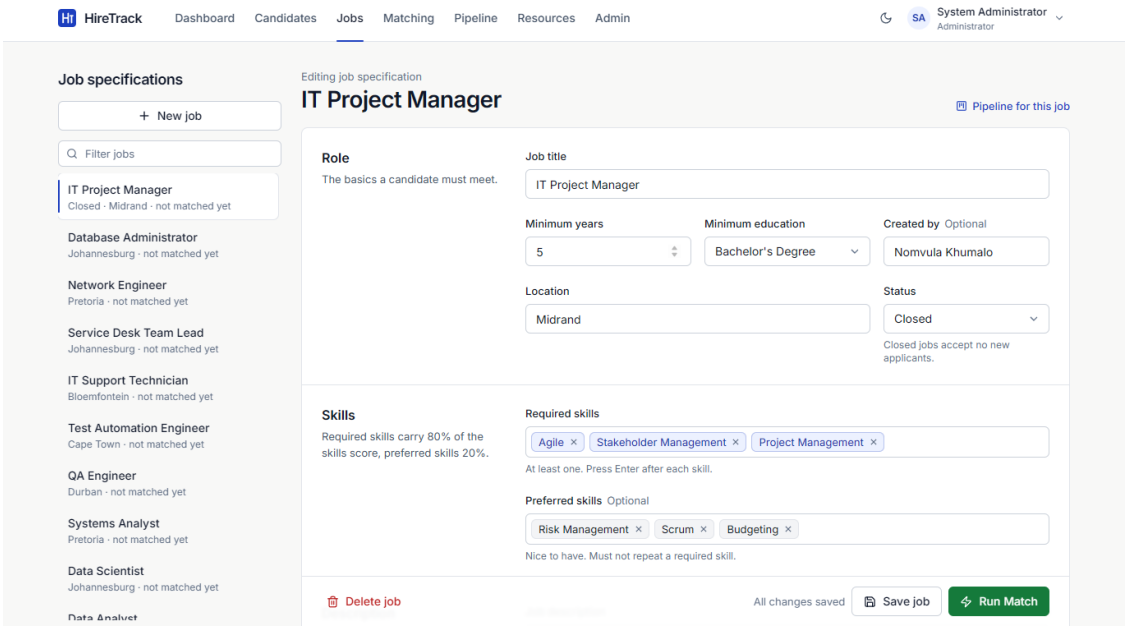


Figure 5.8: Job specification with location, status, skills and weights

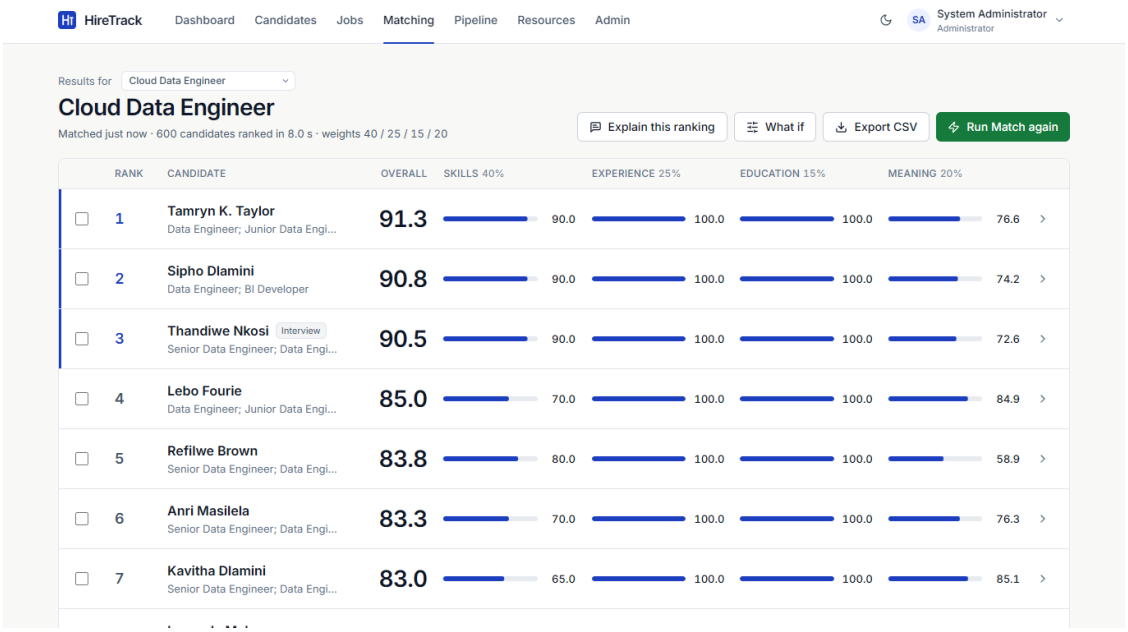


Figure 5.9: Ranked results for Cloud Data Engineer; a pipeline stage badge shows candidates already applied

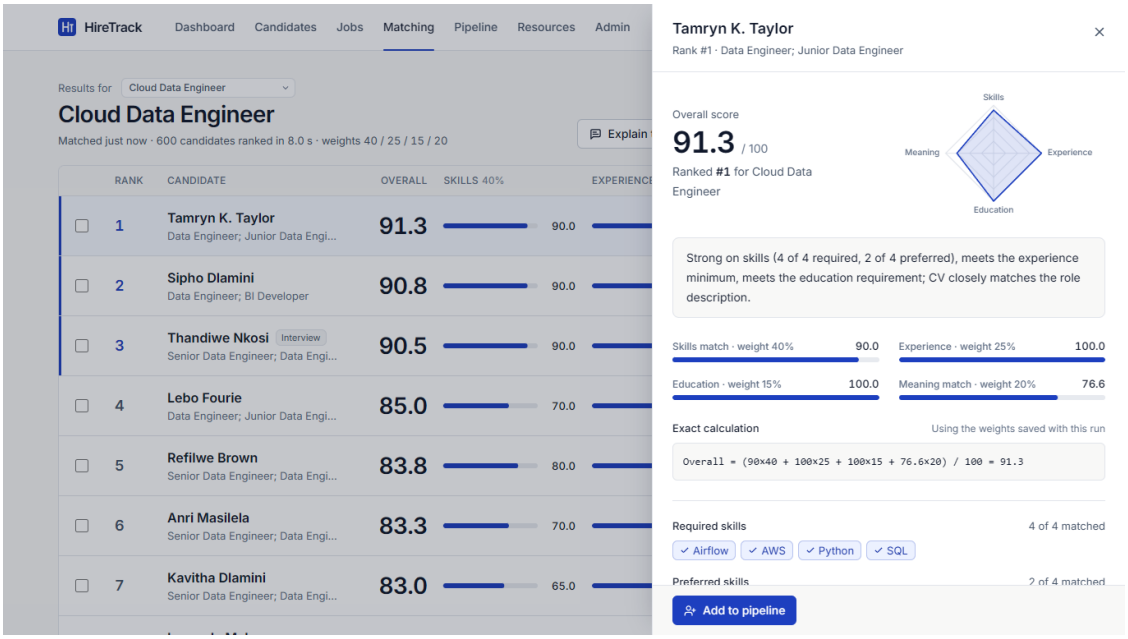


Figure 5.10: Score breakdown with the exact calculation and "Add to pipeline"

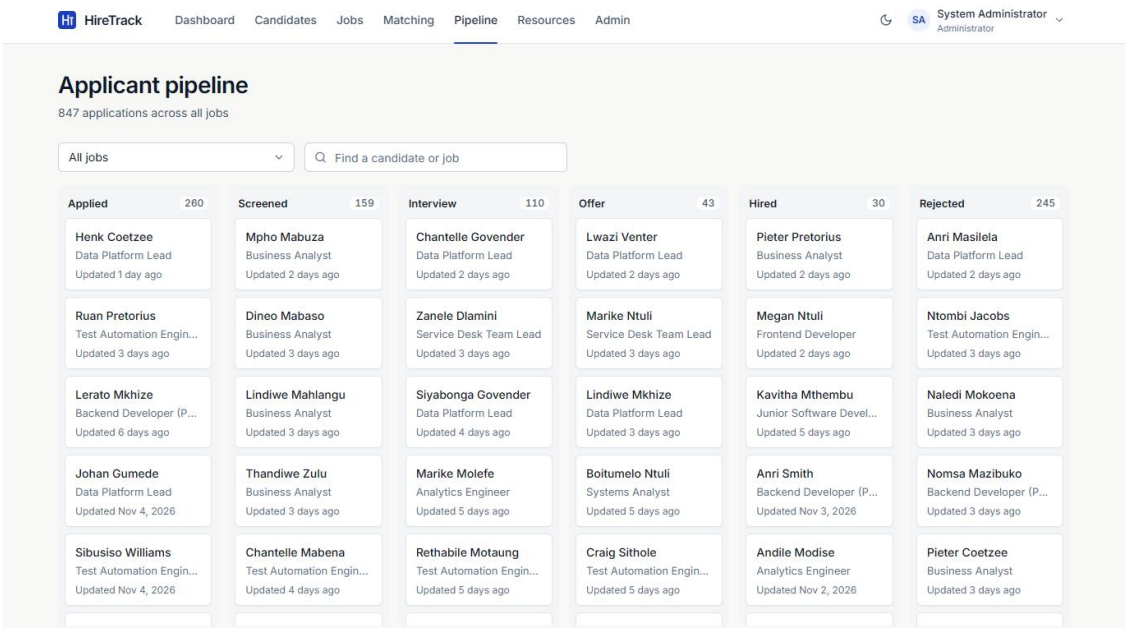


Figure 5.11: Applicant pipeline across all jobs

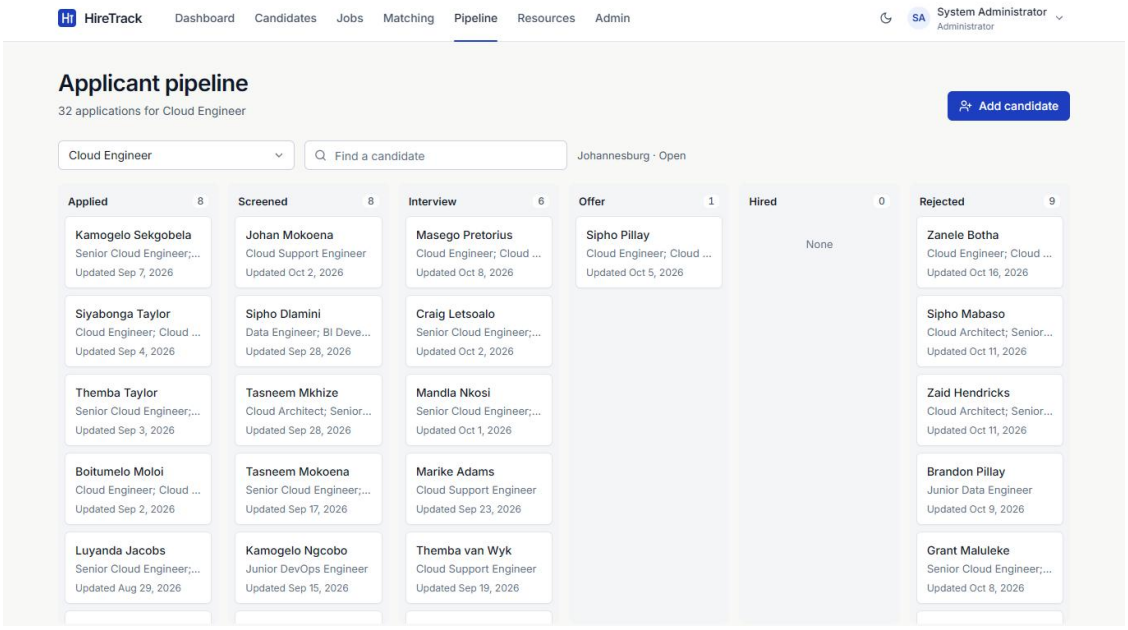


Figure 5.12: Applicant pipeline board for one job

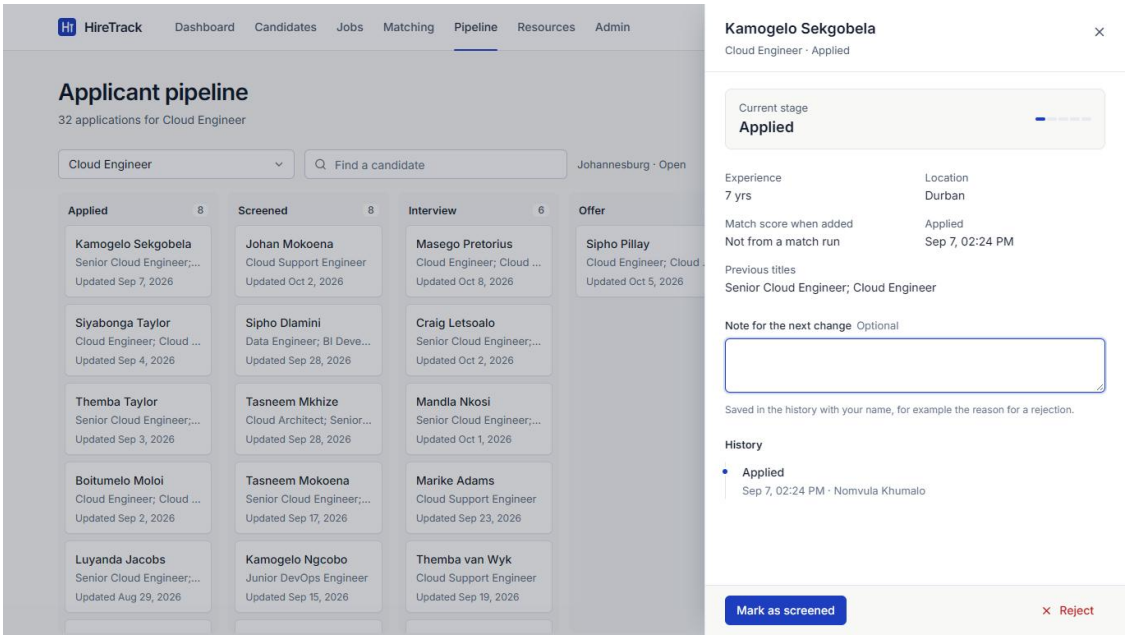


Figure 5.13: Application detail: stage actions, note and audit history

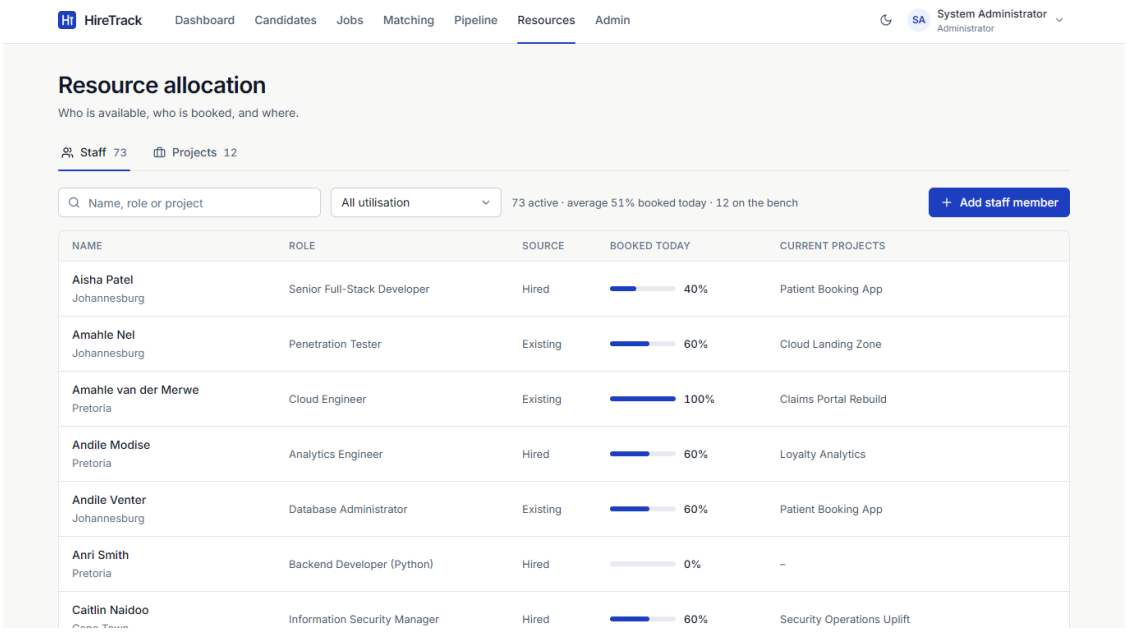


Figure 5.14: Staff utilisation in the Resources screen

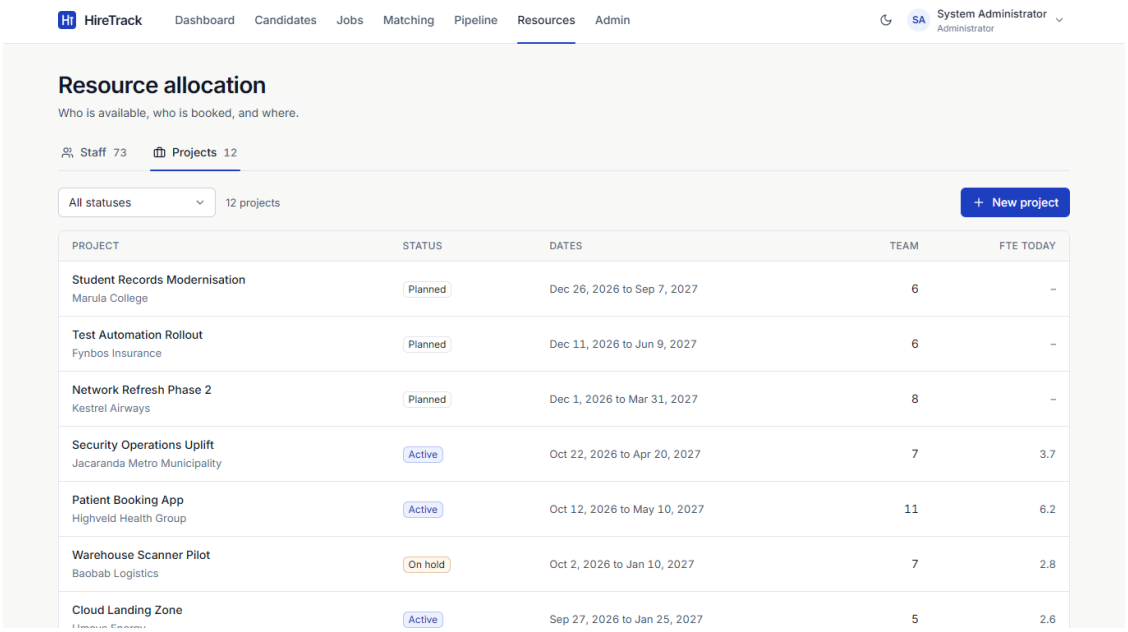


Figure 5.15: Client projects with team size and booked capacity

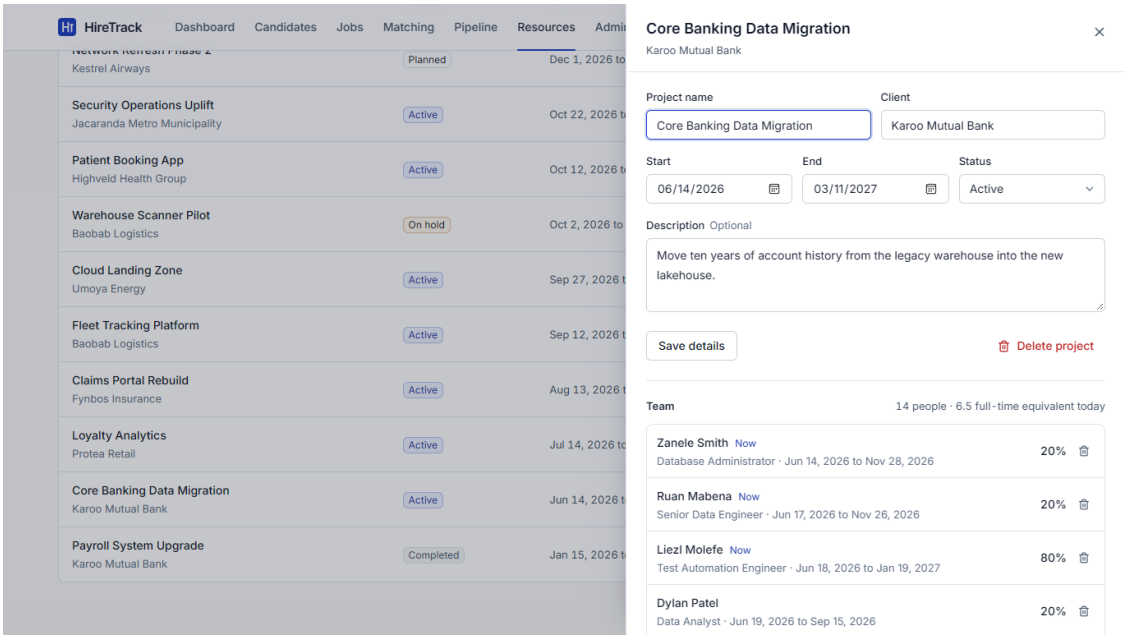


Figure 5.16: Project detail with team allocations

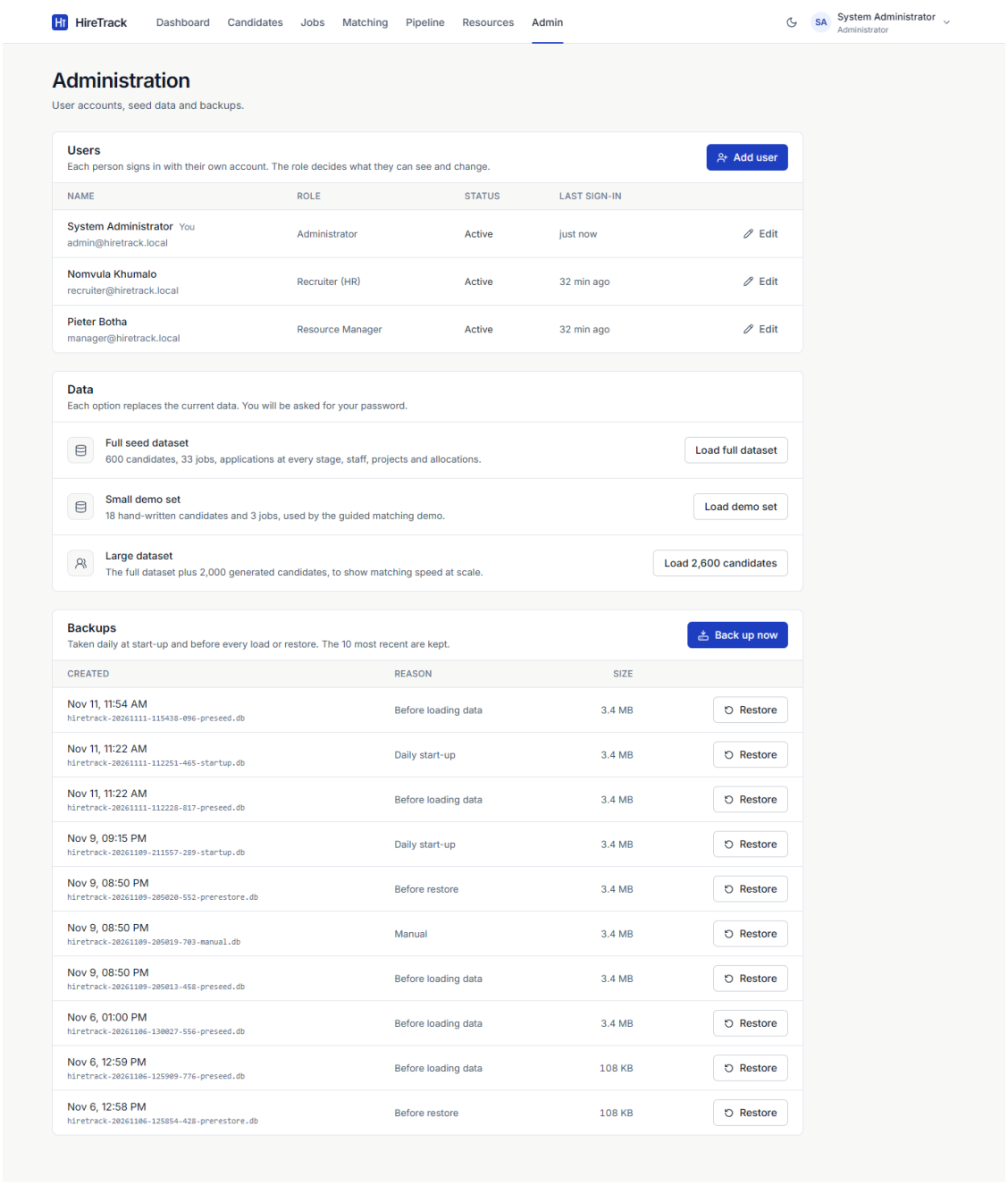


Figure 5.17: Administration: users, datasets and backups

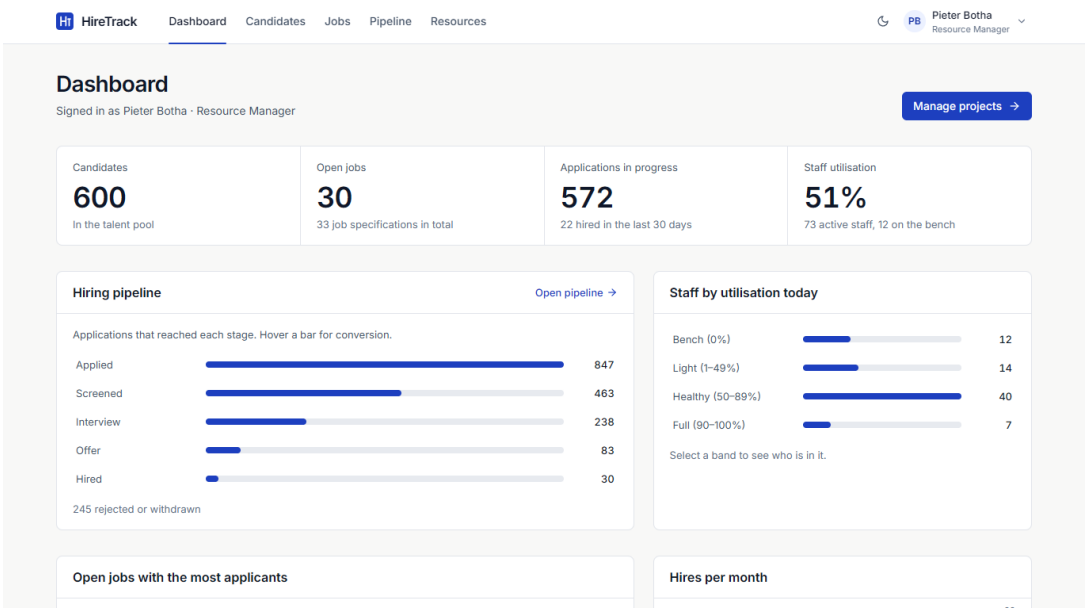


Figure 5.18: Dashboard seen by a Resource Manager (menu limited to that role)

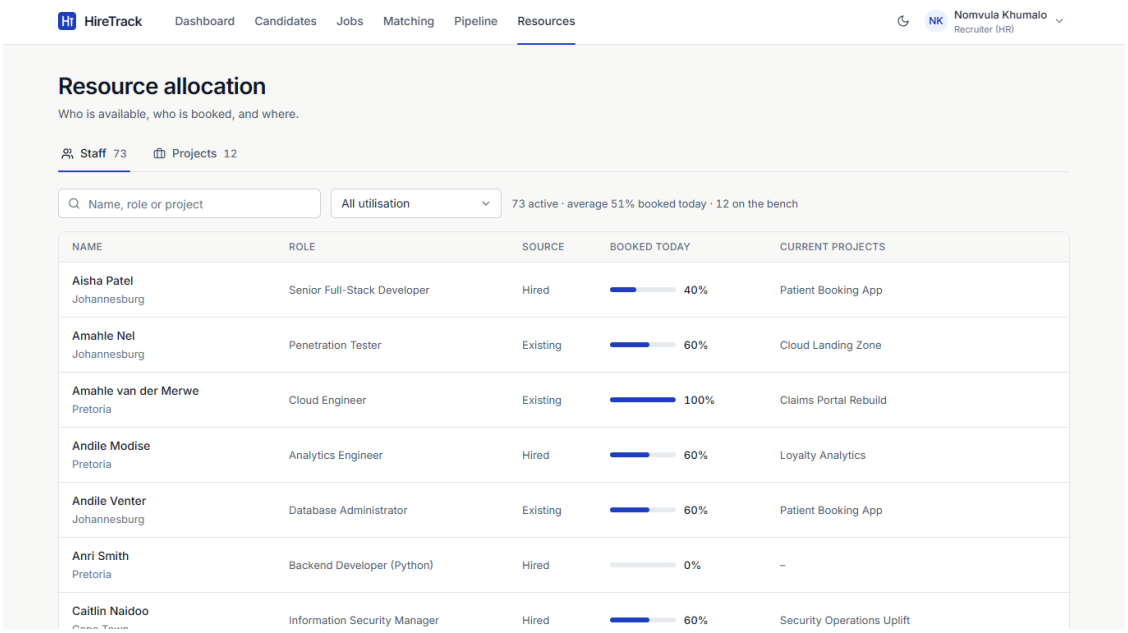


Figure 5.19: A Recruiter sees Resources read-only

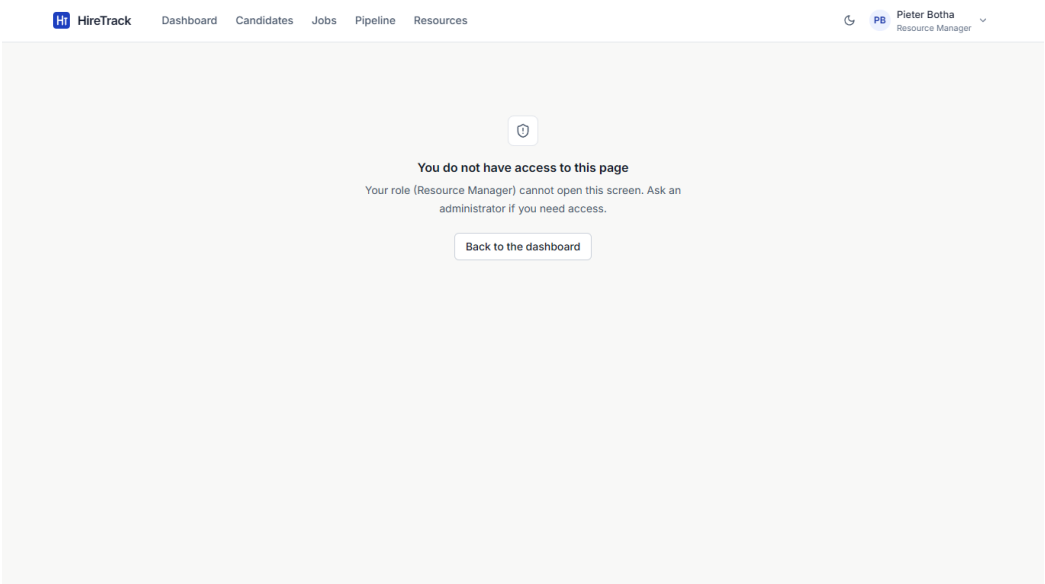


Figure 5.20: A Resource Manager opening the Matching page is refused

5.3 Testing

5.3.1 Approach

Testing followed the test pyramid: many fast unit tests of the rules, a solid layer of API integration tests, component tests of the screens, and a smaller number of end-to-end journeys in a real browser (Section 5.4). All tests are automated and repeatable.

Level	What is tested	Tool	How it is isolated
Unit	Scoring, skill normalisation, pipeline rules, allocation maths, database constraints	pytest	Pure functions; constraint tests use a temporary database
Integration	Every API endpoint, sign-in and roles, import, seed, dashboard, CLI	pytest + FastAPI TestClient	A new temporary data folder per run; every test starts from an empty database with the three accounts
Component	Sign-in, role-based menus, dashboard, pipeline, resources, import, admin, form logic	Vitest + React Testing Library	The API module is replaced with mocks
System / end-to-end	Complete user journeys (TC-01 to TC-22)	Playwright in Edge and Chrome	Own server on port 8010 with a freshly seeded database
Performance	Match speed at 50, 600 and 2,600 candidates	pytest with the real model	HIRETRACK_REAL_MODEL=1

Table 5.6: Testing levels and tools

Fast tests use a deterministic word-hashing embedder instead of the language model, so the 330 backend tests run in under a minute. Password hashing uses 4 bcrypt rounds in tests instead of 12. The performance tests switch the real model back on.

5.3.2 Unit and integration results

Test file	Level	Tests	Passed	Covers
unit/test_scoring.py	Unit	69	69	Four scores, weights, rounding, ranking and tie-breaks, aliases, explanations, fairness
unit/test_pipeline_rules.py	Unit	22	22	Stage order, allowed moves, reopen, funnel
unit/test_allocation_rules.py	Unit	21	21	Overlaps, daily load, peak load, over-allocation messages, bands
unit/test_models.py	Unit	21	21	CHECK, UNIQUE and FK rules; cascades; schema upgrade
integration/test_api.py	Integration	54	54	Candidates, jobs, matching, CSV, backups, seeding, demo
integration/test_auth.py	Integration	64	64	Sign-in, lock-out, tokens, passwords, users, role matrix
integration/test_pipeline_api.py	Integration	23	23	Applications, stage changes, hiring, shortlist from results
integration/test_resources_api.py	Integration	20	20	Staff, projects, allocations, utilisation
integration/test_import.py	Integration	20	20	CSV and Excel import, CV parsing (txt, docx, pdf)

Test file	Level	Tests	Passed	Covers
integration/test_seed_dashboard.py	Integration	9	9	Seed sizes, determinism, legal histories, dashboard
integration/test_cli.py	Integration	4	4	Password reset tool
integration/test_performance.py	Integration	3	3	Speed at 50, 600 and 2,600 candidates
Total backend		330	330	100% pass
frontend src/test (3 files)	Component	49	49	Screens with mocked API, form validation, re-ranking

Table 5.7: Unit, integration and component test results (final run)

5.3.3 Coverage

Backend line coverage is **95%** (2614 of 2748 lines), above the 70% target. The lines not covered are mainly the one-off model download script and error branches that need a missing model or a damaged disk to reach. Frontend line coverage from the component tests is 50.6%; the matching screens are exercised by the end-to-end journeys instead.

Package	Lines	Covered	Coverage
app (core: models, schemas, security, main)	889	827	93%
app/engine	422	392	93%
app/routers	240	240	100%
app/services	1197	1155	96%
Total	2748	2614	95%

Table 5.8: Backend line coverage by package (pytest-cov)

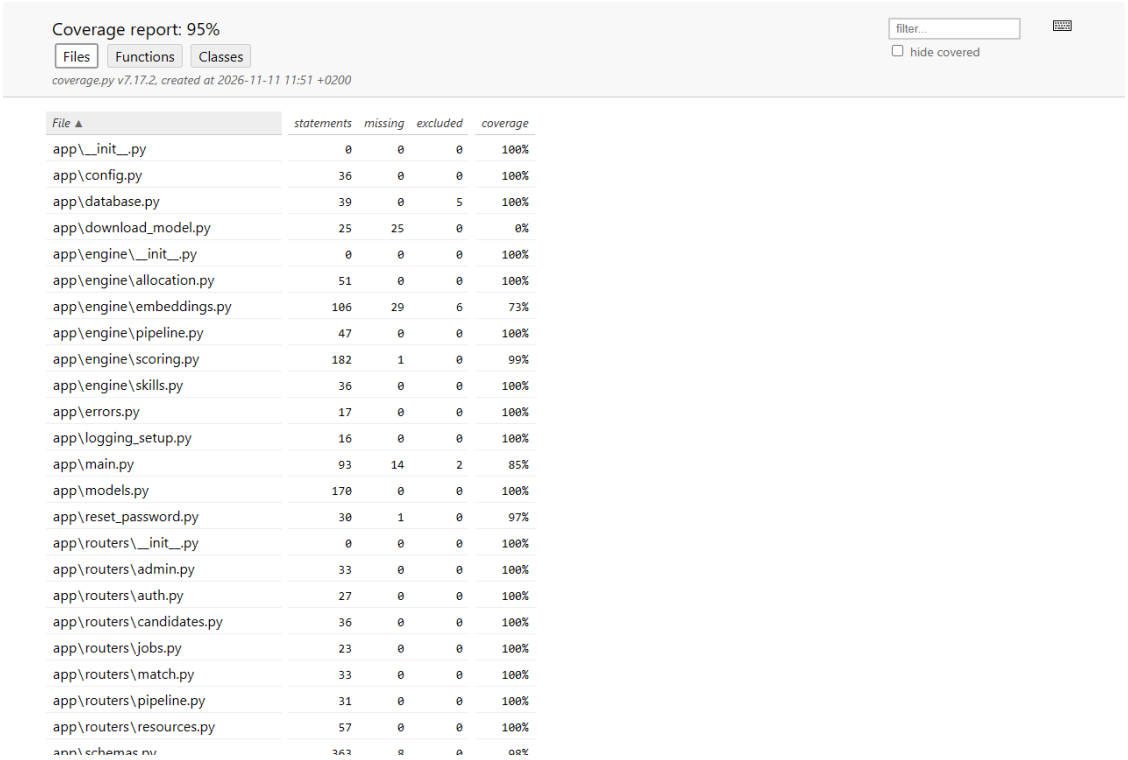


Figure 5.21: pytest-cov HTML report

5.3.4 Performance

Scenario (real all-MiniLM-L6-v2 model, 16-thread laptop CPU)	Time	Target
50 candidates, nothing cached	0.52 s	< 5 s
600 candidates, warm cache (score + save + reload)	0.16 s	< 5 s
600 candidates re-ranked through the screen (Playwright TC-13)	0.69 s	< 5 s
2,600 candidates, warm cache	0.93 to 1.06 s	< 5 s
First match after loading 600 new CVs (each CV read once)	8 to 11 s	one-off
Reading 2,600 new CVs in the background after loading	20.1 s	one-off

Table 5.9: Matching speed

5.3.5 Debugging and code improvement

Testing found seven product defects, all fixed (Section 5.4.4). Two changes improved the code beyond the fixes: the CV parser gained a vocabulary of common skills so it works on an empty database, and the seed data generator was rewritten so stage dates, hires and allocations look like a real company's history. Every fix was followed by a full re-run of all suites.

5.4 System Testing

5.4.1 Approach and requirements

System testing checks the finished system against the requirements, through the screens and the API, as a user would use it. Each test case below is traced to one or more requirements and to the automated test that runs it. The 22 Playwright journeys (TC-01 to TC-22) drive Microsoft Edge against a freshly seeded server; TC-23 to TC-36 check requirements through the API and unit suites; TC-27 and TC-37 were done by hand on a clean copy of the submission.

ID	Requirement
FR1	Add a candidate with name, contact details, location, experience, education, skills, previous titles and CV summary
FR2	View, search, edit and delete candidates
FR3	Create and edit a job specification: title, location, status, skills (required/preferred), minimum experience and education, description
FR4	Adjust the four scoring weights per job; they must total 100%
FR5-FR8	Calculate skills, experience, education and meaning (embedding similarity) scores
FR9	Combine the scores into a weighted overall score and rank every candidate
FR10	Show each candidate's score breakdown, matched and missing skills and the exact calculation
FR11	Seed or reset realistic demonstration data (500+ candidates, 30+ jobs)
FR12	Reject invalid input with a clear message and save nothing
FR13	Export ranked results to CSV
FR14	Sign in with a personal account; three roles (Administrator, Recruiter (HR), Resource Manager) with role-based access
FR15	Administrator manages user accounts; users change their own password
FR16	Capture a CV file (PDF, Word, text) to pre-fill the candidate profile
FR17	Bulk import candidates from CSV or Excel with a per-row error report
FR18	Track each application through Applied, Screened, Interview, Offer, Hired or Rejected under fixed rules
FR19	Keep an audit history of every stage change (who, when, note)
FR20	Add shortlisted candidates from match results to a job's pipeline
FR21	A hired candidate joins the staff pool
FR22	Manage staff and client projects
FR23	Allocate staff to projects by percentage and dates without booking anyone above 100%
FR24	Show utilisation per person and who is on the bench
FR25	Dashboard with key counts and charts
FR26	Back up and restore the database
NFR1	Performance: ranking 500+ candidates returns within 5 seconds
NFR2	Runs on one computer, offline after installation
NFR3	Usability: consistent screens, inline errors, keyboard and screen-reader access
NFR4	Maintainability: business rules in separate, unit-tested modules
NFR5	Privacy: only fictitious data; scoring never uses name, age or gender
NFR6	Reliability: invalid data rejected at screen, API and database level
NFR7	Transparency: every ranking explainable from visible scores

ID	Requirement
NFR8	Compatibility: current Edge and Chrome
NFR9	Security: hashed passwords, signed tokens, role checks on the server, lock-out, confirmed destructive actions
NFR10	Quality: automated test coverage of at least 70% of backend code

Table 5.10: Requirements tested (FR14 to FR26 and NFR9 to NFR10 added in this phase)

Test environment: Windows 11 laptop, 16-thread CPU, no GPU; Python 3.11.18; Node.js 20.16; Microsoft Edge and Google Chrome (current); screen 1366 x 768. Test data: the full seed dataset (600 candidates, 33 jobs, about 850 applications, 73 staff, 12 projects), plus records created by the tests themselves.

5.4.2 Test cases

ID	Req.	Description	Preconditions	Steps	Test data	Expected result	Actual result	Pass/ Fail
TC-01	FR14, NFR9	Sign-in with a wrong password is refused	App running; default accounts exist	1. Open the sign-in page 2. Enter the admin email and a wrong password 3. Select Sign in	admin@hiretrack.local / not-the-password1	Message 'Incorrect email or password'; user stays on the sign-in page	Message shown under the password field; dashboard not opened	Pass
TC-02	FR14	Each role signs in and sees only its menu items	Default accounts exist	1. Sign in as each role in turn 2. Read the main menu 3. Sign out	admin, recruiter, manager accounts	Admin: 7 items incl. Admin. Recruiter: no Admin. Resource Manager: no Matching, no Admin	Menus matched exactly for all three roles	Pass
TC-03	FR14, NFR9	Role checks are enforced by the server, not only hidden in the screen	Signed in as Recruiter	1. Open /admin by URL 2. Call GET /api/users with the recruiter's token	Recruiter token	Screen says 'You do not have access to this page'; API returns 403	Access message shown; API returned 403	Pass
TC-04	FR14	Signing out ends the session	Signed in as Resource Manager	1. Open the account menu, select Sign out 2. Open /resources by URL	-	Sign-in page shown again	Sign-in page shown	Pass
TC-05	FR1, FR12, NFR6	Invalid candidate data is refused with a message under each field	Signed in as Recruiter	1. Candidates > Add candidate 2. Leave name, education, skills empty; years = -2 3. Select Add candidate	years_experience = -2	Four inline messages; nothing saved; count stays 600	'Full name is required', 'cannot be less than 0', 'Choose an education level', 'Add at least one skill'; count 600	Pass
TC-06	FR1, FR2	Add, search, edit and delete a candidate	Signed in as Recruiter; 600 candidates	1. Add a candidate with skills Python and K8s 2. Search by name 3. Edit years to 5 4. Delete and confirm	Name 'Zinhle Test <id>', Durban, 4 years, Bachelor's	Count 601 after add; search shows 1 of 601; edit saved; count back to 600 after delete	As expected; K8s stored as kubernetes and shown as typed	Pass
TC-07	FR16	A CV file pre-fills the candidate form	Add Candidate panel open	1. Select Fill from CV 2. Choose a .txt CV	CV with name, email, phone and 'Airflow pipelines in Python and SQL on AWS'	Name, email, phone, summary filled; skills added as chips; nothing saved yet	All fields filled; chips Airflow, Python, SQL, AWS. (First run: chips were lower case - defect D1, fixed)	Pass
TC-08	FR17	Bulk CSV import keeps valid rows and reports bad rows	Signed in as Recruiter	1. Import CSV or Excel 2. Choose a file with 2 valid rows and 1 row with years = -1 3. Import file	3-row CSV	2 imported, 1 error 'row 4, Years, Cannot be less than 0'; count 602	Imported 2, errors table showed row 4; count 602	Pass
TC-09	FR3, FR4	Create a job; Run Match is locked until the weights total 100%	Signed in as Recruiter	1. Jobs > New job, fill title, location, skills, description 2. Set skills weight to 35 (total 95%) 3. Set it back to 40 4. Save	Python Developer, Cape Town, 3 years	At 95% a warning shows and Run Match is disabled; at 100% it is enabled; job saved	Warning '(now 95%)' and disabled button; enabled at 100%; saved	Pass

ID	Req.	Description	Preconditions	Steps	Test data	Expected result	Actual result	Pass/ Fail
TC-10	FR5-FR8, FR9, FR10, NFR7	Run a match and read the breakdown	Full seed data loaded	1. Open Cloud Data Engineer 2. Run Match 3. Open the top candidate	600+ candidates	All candidates ranked; breakdown shows four scores, skills and 'Overall = (...) / 100 = x'	602 ranked; top 50 shown; calculation shown and reproduces the overall score	Pass
TC-11	FR4, FR9, NFR7	What-if weights re-rank without saving	Results open	1. What if 2. Set Meaning to 60%	semantic weight 60	List re-ranks; 'Showing what-if ranking' label; saved run unchanged	Re-ranked instantly with the label shown	Pass
TC-12	FR13, FR20	Shortlist into the pipeline and export CSV	Results open	1. Tick 2 candidates not yet in the pipeline 2. Add to pipeline 3. Export CSV	2 candidates	Toast '2 candidates added'; CSV named hiretrack-cloud-data-engineer-<date>.csv with 600+ rows	As expected; CSV header and 603 lines checked	Pass
TC-13	NFR1	Re-running a 600-candidate match through the screen takes under 5 s	Model loaded, CVs cached	1. Results > Run Match again 2. Time until 'Match complete'	602 candidates	Under 5 seconds	0.68 s end to end (Edge)	Pass
TC-14	FR18, FR19, FR21	Move an applicant from Applied to Hired with notes	Candidate applied to Cloud Engineer	1. Open the card 2. For each stage add a note and select the next action	Notes for each move	Stage advances one step at a time; history shows 5 entries with notes and user; person appears in Staff as Hired	As expected	Pass
TC-15	FR18, FR19	Stages cannot be skipped; a rejection can be reopened	An application at Applied	1. POST stage=offer 2. Reject with a note 3. Reopen	-	API 422 'next stage is Screened'; rejected then back at Applied; note kept in history	As expected	Pass
TC-16	FR14, FR18	A Resource Manager can view but not change the pipeline	Signed in as Resource Manager	1. Pipeline 2. Open any card	-	History visible; no stage buttons	No action buttons shown; API also returns 403	Pass
TC-17	FR22, FR23, FR24	Create a project, allocate staff, block over-allocation	Signed in as Resource Manager	1. Add staff member 2. New project (active) 3. Allocate 70% 4. Try another 50% on the same dates	70% then 50%	First saved (0.7 FTE); second refused: 'most you can add for these dates is 30%'; staff list shows 70%	As expected	Pass
TC-18	FR22, FR12	Project end date before start date is refused	New project panel	1. Start = today+30, End = today+1 2. Create project	-	'End date cannot be before the start date'; nothing saved	As expected	Pass
TC-19	FR24	Bench filter shows only unallocated staff	Seed data loaded	1. Open /resources?band=bench	-	Every row shows 0%	All rows 0%	Pass

ID	Req.	Description	Preconditions	Steps	Test data	Expected result	Actual result	Pass/ Fail
TC-20	FR25, NFR3	Dashboard shows key counts and charts; bars are usable by keyboard	Seed data loaded	1. Sign in as Admin 2. Check tiles and charts 3. Select the Bench bar	-	4 tiles and 5 chart panels; selecting Bench opens the filtered staff list	As expected. (First run: bars were hidden from keyboard/screen readers - defect D2, fixed)	Pass
TC-21	FR15	An administrator creates a user who can then sign in	Signed in as Admin	1. Admin > Add user (Recruiter) 2. Sign out 3. Sign in as the new user	hr.<id>@hiretrack.local / Welcome123	User created; sign-in succeeds with the Recruiter menu	As expected	Pass
TC-22	FR26, FR11, NFR9	Manual backup; data reset needs the admin's password	Signed in as Admin	1. Back up now 2. Load demo set with a wrong password 3. Cancel	wrong-password1	Backup listed; 'Incorrect password'; data unchanged	As expected	Pass
TC-23	FR11	Seed script builds the realistic dataset	Empty database	1. python seed/seed_database.py	-	600 candidates, 33 jobs (30 open), 500+ applications with legal histories, staff, 12 projects, no one above 100%	600 candidates, 33 jobs, 847 applications, 1,906 events, 73 staff, 12 projects, 102 allocations; all histories legal	Pass
TC-24	FR26	Restore refuses a corrupt or unknown backup file	Admin	1. Restore a damaged file 2. Restore a path with ../	corrupt .db; '../../data/hiretrack.db'	Integrity check fails, nothing restored; traversal refused	As expected	Pass
TC-25	FR15, NFR9	There is always one active administrator; users cannot deactivate themselves	Admin	1. Change the only admin to Recruiter 2. Deactivate own account	-	Both refused with a reason	As expected	Pass
TC-26	NFR1	Ranking speed with the real model	Real all-MiniLM-L6-v2 model	Run tests/integration/test_performance.py with HIRETRACK_REAL_MODEL=1	50, 600 and 2,600 candidates	Each warm match under 5 s	50 uncached: 0.52 s; 600 warm: 0.16 s; 2,600 warm: 0.93-1.06 s; one-off encoding of 2,600 CVs 20.1 s	Pass
TC-27	NFR2	The app runs with no internet connection	Fresh install from a copy of the repository	1. Start the server with HF_HUB_OFFLINE=1 and an unreachable proxy 2. Sign in and run a match	HTTP(S)_PROXY=http://127.0.0.1:9	Everything works; no outbound connection needed	Match ran and results loaded with all network access blocked	Pass
TC-28	FR5-FR8, FR9, NFR5, NFR7	Scoring ignores personal details	-	Score two candidates identical except name and email	-	Identical scores; name used only as the final tie-break	As expected	Pass
TC-29	NFR6	Database constraints reject bad data written directly	-	Insert bad rows through the ORM, bypassing the API	years -1, weights 110%, allocation 0%, duplicate application	IntegrityError for every case	As expected (21 cases)	Pass

ID	Req.	Description	Preconditions	Steps	Test data	Expected result	Actual result	Pass/ Fail
TC-30	NFR9	Five failed sign-ins lock the account for 60 s	-	1. Five wrong passwords 2. Correct password	-	429 'Too many failed sign-in attempts'	As expected	Pass
TC-31	NFR9	Expired or forged tokens are refused	-	Call /api/auth/me with an expired and a garbage token	-	401 with 'expired' / 'sign in'	As expected	Pass
TC-32	NFR9	Passwords are stored only as bcrypt hashes	-	Read app_user.password_hash	-	Every value starts with $2b$; no plain text	As expected	Pass
TC-33	NFR6, FR12	Search text cannot inject SQL	Candidates exist	Search for x' OR '1'='1	-	No rows matched; no error	As expected	Pass
TC-34	NFR8	Full journey suite in a second browser	Chrome installed	HT_BROWSER=chrome npx playwright test	22 journeys	All pass	22 of 22 passed in Chrome	Pass
TC-35	NFR10	Backend coverage at least 70%	-	pytest --cov=app	-	>= 70%	95% line coverage	Pass
TC-36	NFR4	Business rules can be tested without a database	-	Run tests/unit (scoring, pipeline, allocation)	-	All pass with no server or database	133 unit tests passed	Pass
TC-37	Installation	Fresh install from a copy of the repository on a clean folder	Python 3.11, Node 20	Clone, run start.bat, sign in	-	App installs, seeds and opens at http://127.0.0.1:<port>	See Installation section: completed and verified	Pass

Table 5.11: System test cases (also in system_testing/Test_Cases.xlsx with the automated test for each case)

5.4.3 Evaluation of the testing results

Pass rate. All 37 system test cases passed on the final runs: 22 of 22 end-to-end journeys in Edge and 22 of 22 in Chrome, 330 of 330 backend tests and 49 of 49 frontend tests, 401 automated tests in total. Every requirement from FR1 to FR26 and NFR1 to NFR10 is covered by at least one passing case (traceability sheet in Test_Cases.xlsx).

Not first-time results. The first end-to-end run passed 14 of 22 (64%); the first backend run passed 322 of 326. Each failure was investigated and classified as a product defect, a test defect or a test-environment issue. Raw outputs of every run are kept in testing/results.

Suite	First run	Final run
Backend (pytest)	322 / 326	330 / 330
Frontend (Vitest)	45 / 46	49 / 49
End-to-end, Edge (Playwright)	14 / 22	22 / 22
End-to-end, Chrome (Playwright)	not run	22 / 22

Table 5.12: Pass rate from first to final run

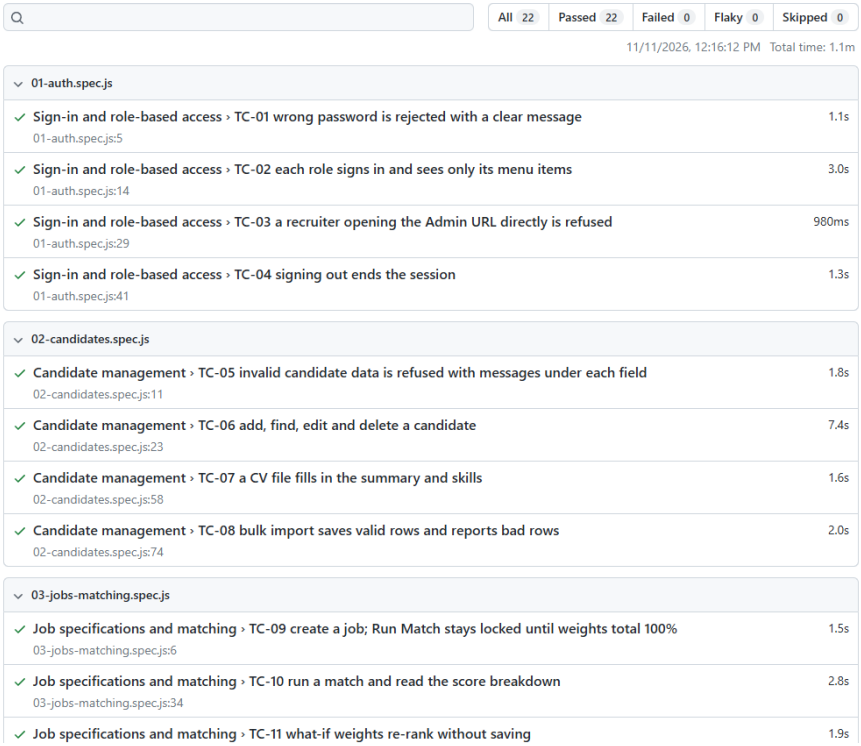


Figure 5.22: Playwright HTML report of the final end-to-end run

5.4.4 Defects found and how they were fixed

ID	Found by	Defect	Fix
D1	TC-07	Skills read from a CV appeared as lower-case chips	Converted to display labels before adding them to the form
D2	TC-20	Clickable dashboard bars were hidden from keyboard and screen-reader users	Made real buttons with names; component test added
D3	test_import	CV parsing found no skills on a new, empty database	Built-in vocabulary of common skills plus the alias table

ID	Found by	Defect	Fix
D4	Screenshot review	Seed data unrealistic: same timestamps, hires with other open offers, 59% of staff on the bench	Dates spread over time, a hire withdraws other applications, bench about 1 in 8
D5	Screenshot review	"170% booked today" on a project read as over-booking	Shown as full-time equivalents (1.7 FTE)
D6	Manual test	start.bat busy-port check never fired	Replaced the regular expression with plain findstr filters
D7	Fresh install	start.bat could not find Python 3.11 installed outside PATH	PYTHON override added to both start scripts and documented

Table 5.13: Product defects

Nine test defects were also fixed (for example ambiguous locators, an expected count that ignored records created by an earlier test, and an expectation of 29 open jobs where 30 is correct). Two test-environment issues were found. First, the end-to-end server had switched off the model warm-up the real app performs at start-up, so the first match was slow. It now starts exactly like the real app. Second, under automation Microsoft Edge holds a finished blob download open, which hung the next test. The CSV test now captures the file at the last browser step in Edge, while Chrome performs the full save.

5.4.5 Limitations

- Firefox and Safari were not tested; Edge and Chrome were.
- Usability was judged by review and accessibility checks, not by observing real recruiters.
- The first match after loading new CVs takes longer than 5 seconds because each CV is read once by the model; later matches take under a second.
- SQLite and a server bound to 127.0.0.1 suit one user at a time; concurrent load was not tested.
- Frontend component coverage is 50.6%; the older matching screens rely on end-to-end tests.
- Scanned (image-only) CVs cannot be read; there is no OCR.
- Seed data is generated, so match quality on real CVs has not been measured.

5.4.6 Recommendations

1. Add Firefox and other screen sizes to the Playwright run.
2. Hold a short usability session with two or three recruiters, using the test journeys as tasks.
3. Move to PostgreSQL and a lab-network server before several recruiters use it at once, and add a load test.
4. Add component tests for the Candidates, Jobs and Results screens.
5. Encode CVs in the background as soon as they are imported or edited, so the first match is never cold.
6. Add OCR for scanned CVs and compare match rankings with recruiter shortlists.

5.5 Installation

5.5.1 Summary

HireTrack installs on one Windows or macOS computer with a single command. The full, step-by-step guide with manual installation, test commands and troubleshooting is INSTALLATION_GUIDE.md in the repository.

Prerequisite	Version
Python	3.11 (64-bit)
Node.js and npm	18 or later (tested with 20.16)
Browser	Microsoft Edge or Google Chrome
Disk / memory	about 3 GB free; 4 GB RAM minimum, 8 GB recommended
Internet	During installation only

Table 5.14: Prerequisites

5.5.2 Steps

1. Clone the repository and open it: `git clone https://github.com/RohanRajaramNaik/hiretrack.git` then `cd hiretrack`. (Alternatively use Code, Download ZIP on the repository page.)
2. Windows: run `start.bat`. macOS/Linux: run `chmod +x start.sh` then `./start.sh`.
3. The first run creates the Python virtual environment and installs the packages, downloads the language model (about 90 MB), builds the screens, and creates and seeds the database. Later runs start in a few seconds and work offline.
4. Open `http://127.0.0.1:8000` (the script opens it) and sign in with one of the accounts below.

Role	Email	Password
Administrator	admin@hiretrack.local	Admin@123
Recruiter (HR)	recruiter@hiretrack.local	Recruiter@123
Resource Manager	manager@hiretrack.local	Manager@123

Table 5.15: Default accounts (change the passwords before real use)

Options: `PORT` (another port), `PYTHON` (path to Python 3.11 if not on PATH) and `HT_NO_BROWSER`. The database can be re-seeded at any time with `python seed/seed_database.py` or from the Admin screen.

5.5.3 Verification on a clean copy

The guide was verified by doing a fresh installation from a copy of the repository contents in a new temporary folder on the test laptop, with no virtual environment, no node_modules, no model and no database. start.bat was run on port 8002. It installed the packages, downloaded the model, built the frontend, seeded 600 candidates and 33 jobs, and started the server. All three accounts signed in, a match ran, and the dashboard, pipeline and resources screens loaded. The server was then restarted with HF_HUB_OFFLINE=1 and all HTTP and HTTPS traffic sent to an unreachable proxy. Matching and every screen still worked, which confirms NFR2 (offline after installation). The install log is in testing/results/fresh-install-log.txt.

Conclusion

The Implementation Phase delivered a complete, working HireTrack. The matching engine from the Design phase is built exactly to its specification and explains every score. Around it, the system now has the remaining objectives of the Proposal: personal accounts with three roles, an applicant pipeline with a full audit history, resource allocation that cannot over-book anyone, bulk import and CV capture, and a dashboard. The database seeds itself with 600 candidates, 33 jobs and a realistic hiring and project history.

The system was tested at four levels with 401 automated tests and 37 traced system test cases. Every requirement passed on the final runs, with 95% backend coverage, and a 600-candidate match takes well under one second once CVs are cached. Testing found and fixed seven product defects; the remaining limitations (browser range, cold first match, single-user database) are documented with recommendations. A fresh installation from the submitted files was verified with one command.

References

FastAPI (2026) FastAPI documentation. Available at: https://fastapi.tiangolo.com (Accessed: 6 November 2026).

Jones, M., Bradley, J. and Sakimura, N. (2015) JSON Web Token (JWT), RFC 7519. Internet Engineering Task Force.

Meta Platforms (2026) React documentation. Available at: https://react.dev (Accessed: 6 November 2026).

Microsoft (2026) Playwright documentation. Available at: https://playwright.dev (Accessed: 10 November 2026).

OWASP Foundation (2021) OWASP Top 10. Available at: https://owasp.org/Top10/ (Accessed: 6 November 2026).

Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: Sentence embeddings using Siamese BERT-networks', Proceedings of EMNLP-IJCNLP 2019, pp. 3982-3992.

Republic of South Africa (2013) Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer.

Richfield Graduate Institute of Technology (2026) IT Project Guidelines, Semester 2 2026. Faculty of Information Technology.

SQLAlchemy (2026) SQLAlchemy 2.0 documentation. Available at: https://docs.sqlalchemy.org (Accessed: 6 November 2026).

SQLite (2026) SQLite online backup API. Available at: https://www.sqlite.org/backup.html (Accessed: 6 November 2026).

Schwalbe, K. (2019) Information Technology Project Management. 9th edn. Boston: Cengage Learning.

Appendix A: Core Backend Source Code

Full listing of the core backend files, copied from the repository (https://github.com/RohanRajaramNaik/hiretrack) at the time of submission. Line numbers are those of each file.

backend/app/main.py

Application entry point: routers, middleware, static serving.

```python
"""HireTrack FastAPI application.

Development:  uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload   (Vite on 5173)
Production:   npm run build in frontend/, then the same uvicorn command serves
              the built frontend and the API on http://127.0.0.1:8000
"""
from __future__ import annotations

import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from sqlalchemy.exc import IntegrityError
from starlette.exceptions import HTTPException as StarletteHTTPException

from .config import settings
from .database import init_db
from .errors import AppError
from .logging_setup import setup_logging
from .routers import admin, auth, candidates, jobs, match, pipeline, resources

log = logging.getLogger("hiretrack")

FIELD_LABELS = {
    "full_name": "Full name",
    "resume_summary": "Resume summary",
    "job_title": "Job title",
    "job_description": "Job description",
    "years_experience": "Years of experience",
    "min_experience": "Minimum experience",
    "level_id": "Education level",
    "min_level_id": "Minimum education",
    "location": "Location",
    "role_title": "Role",
    "allocation_percent": "Allocation",
    "start_date": "Start date",
    "end_date": "End date",
    "name": "Name",
    "client": "Client",
    "password": "Password",
    "email": "Email",
}


@asynccontextmanager
async def lifespan(_app: FastAPI):
    setup_logging()
    settings.ensure_dirs()
    init_db()
    if not settings.disable_startup_backup:
        from .services import backup_service, seed_service

        try:
            backup_service.startup_backup()
        except Exception:  # pragma: no cover
            log.exception("Start-up backup failed")
        seed_service.warm_embeddings_async()
    log.info("HireTrack started on %s:%s", settings.host, settings.port)
    yield


app = FastAPI(title="HireTrack: Applicant Tracking & Resource Allocation", version="2.0.0", lifespan=lifespan)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["Content-Type", "Authorization", "X-Confirm-Password"],
)


# ---------------------------------------------------------------- errors
def _friendly(err: dict) -> tuple[str, str | None]:
    """Turn one Pydantic error into (message, field) in plain English."""
    loc = [str(p) for p in err.get("loc", []) if p not in ("body", "query", "path")]
    field = loc[0] if loc else None
    ctx = err.get("ctx") or {}
    original = ctx.get("error")
    if original is not None and getattr(original, "field", None):
        return str(original), original.field
    label = FIELD_LABELS.get(field or "", (field or "Value").replace("_", " ").capitalize())
    t = err.get("type", "")
    if t == "missing":
```

```
        return f"{label} is required", field
    if t == "string_too_short":
        n = ctx.get("min_length")
        return (f"{label} is required" if n == 1 else f"{label} must be at least {n} characters"), field
    if t == "string_too_long":
        return f"{label} must be {ctx.get('max_length')} characters or fewer", field
    if t in ("greater_than_equal", "greater_than"):
        return f"{label} cannot be less than {ctx.get('ge', ctx.get('gt'))}", field
    if t in ("less_than_equal", "less_than"):
        return f"{label} cannot be more than {ctx.get('le', ctx.get('lt'))}", field
    msg = err.get("msg", "Invalid value")
    return msg.removeprefix("Value error, "), field


@app.exception_handler(RequestValidationError)
async def _validation_handler(_req: Request, exc: RequestValidationError):
    errors = exc.errors()
    detail, field = _friendly(errors[0]) if errors else ("Invalid request", None)
    return JSONResponse(status_code=422, content={"detail": detail, "field": field})


@app.exception_handler(AppError)
async def _app_error_handler(_req: Request, exc: AppError):
    return JSONResponse(status_code=exc.status_code, content={"detail": exc.detail, "field": exc.field})


@app.exception_handler(IntegrityError)
async def _integrity_handler(_req: Request, exc: IntegrityError):
    log.warning("Integrity error: %s", exc.orig)
    return JSONResponse(status_code=409, content={"detail": "That change conflicts with existing data", "field": None})


@app.exception_handler(StarletteHTTPException)
async def _http_handler(_req: Request, exc: StarletteHTTPException):
    return JSONResponse(status_code=exc.status_code, content={"detail": str(exc.detail), "field": None})


# ---------------------------------------------------------------------- routes
app.include_router(auth.router)
app.include_router(candidates.router)
app.include_router(jobs.router)
app.include_router(match.router)
app.include_router(admin.router)
app.include_router(pipeline.router)
app.include_router(resources.router)


@app.get("/api/health")
def health() -> dict:
    return {"status": "ok"}


@app.api_route("/api/{rest:path}", methods=["GET", "POST", "PUT", "DELETE"], include_in_schema=False)
def api_not_found(rest: str):
    """Unknown API paths always get the standard error shape, whatever the method."""
    return JSONResponse(status_code=404, content={"detail": "Not found", "field": None})


# ------------------------------------------------------- built frontend (SPA)
_dist = settings.frontend_dist
if (_dist / "index.html").exists():
    if (_dist / "assets").exists():
        app.mount("/assets", StaticFiles(directory=_dist / "assets"), name="assets")

    @app.get("/{full_path:path}", include_in_schema=False)
    def spa(full_path: str):
        if full_path.startswith("api/"):
            return JSONResponse(status_code=404, content={"detail": "Not found", "field": None})
        candidate = (_dist / full_path).resolve()
        if full_path and candidate.is_file() and _dist.resolve() in candidate.parents:
            return FileResponse(candidate)
        return FileResponse(_dist / "index.html")
```

backend/app/models.py

Database tables and constraints.

```
"""ORM models (3NF). CHECK constraints mirror the validation rules exactly."""
from __future__ import annotations

from datetime import date, datetime, timezone

from sqlalchemy import (
    JSON,
    Boolean,
    CheckConstraint,
    Date,
    DateTime,
    Float,
    ForeignKey,
    Index,
    Integer,
    String,
    Text,
    UniqueConstraint,
)
from sqlalchemy.orm import Mapped, mapped_column, relationship

from .database import Base


```

```python
def utcnow() -> datetime:
    return datetime.now(timezone.utc).replace(tzinfo=None)


class EducationLevel(Base):
    __tablename__ = "education_level"
    level_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    level_name: Mapped[str] = mapped_column(String(40), unique=True, nullable=False)
    __table_args__ = (CheckConstraint("level_id BETWEEN 1 AND 7", name="ck_level_range"),)


class Skill(Base):
    __tablename__ = "skill"
    skill_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    # Stored lowercase + trimmed (enforced by SkillNormaliser and the CHECK below).
    name: Mapped[str] = mapped_column(String(60), unique=True, nullable=False)
    __table_args__ = (
        CheckConstraint("name = lower(trim(name)) AND length(name) > 0", name="ck_skill_normalised"),
    )


class Candidate(Base):
    __tablename__ = "candidate"
    candidate_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    full_name: Mapped[str] = mapped_column(String(100), nullable=False)
    email: Mapped[str | None] = mapped_column(String(254), nullable=True)
    phone: Mapped[str | None] = mapped_column(String(30), nullable=True)
    location: Mapped[str | None] = mapped_column(String(80), nullable=True)
    years_experience: Mapped[float] = mapped_column(Float, nullable=False)
    level_id: Mapped[int] = mapped_column(ForeignKey("education_level.level_id"), nullable=False)
    previous_titles: Mapped[str | None] = mapped_column(String(500), nullable=True)
    resume_summary: Mapped[str] = mapped_column(Text, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, nullable=False)

    level: Mapped[EducationLevel] = relationship(lazy="joined")
    skills: Mapped[list["CandidateSkill"]] = relationship(
        back_populates="candidate", cascade="all, delete-orphan", passive_deletes=True
    )

    __table_args__ = (
        CheckConstraint("length(trim(full_name)) BETWEEN 2 AND 100", name="ck_cand_name"),
        CheckConstraint("years_experience BETWEEN 0 AND 50", name="ck_cand_years"),
        CheckConstraint("length(resume_summary) BETWEEN 20 AND 3000", name="ck_cand_summary"),
        Index("ix_candidate_name", "full_name"),
    )


class CandidateSkill(Base):
    __tablename__ = "candidate_skill"
    candidate_id: Mapped[int] = mapped_column(
        ForeignKey("candidate.candidate_id", ondelete="CASCADE"), primary_key=True
    )
    skill_id: Mapped[int] = mapped_column(ForeignKey("skill.skill_id", ondelete="CASCADE"), primary_key=True)
    # What the user actually typed (e.g. "JS" for javascript). Lets the results
    # screen explain alias matches. Depends on the full key, so 3NF holds.
    entered_as: Mapped[str | None] = mapped_column(String(60), nullable=True)

    candidate: Mapped[Candidate] = relationship(back_populates="skills")
    skill: Mapped[Skill] = relationship(lazy="joined")


class JobSpecification(Base):
    __tablename__ = "job_specification"
    job_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    job_title: Mapped[str] = mapped_column(String(100), nullable=False)
    created_by: Mapped[str | None] = mapped_column(String(100), nullable=True)
    location: Mapped[str] = mapped_column(String(80), nullable=False, default="Johannesburg")
    # open = accepting applicants; closed = filled or withdrawn
    status: Mapped[str] = mapped_column(String(10), nullable=False, default="open")
    min_experience: Mapped[float] = mapped_column(Float, nullable=False, default=0)
    min_level_id: Mapped[int] = mapped_column(ForeignKey("education_level.level_id"), nullable=False)
    job_description: Mapped[str] = mapped_column(Text, nullable=False)
    w_skills: Mapped[int] = mapped_column(Integer, nullable=False, default=40)
    w_experience: Mapped[int] = mapped_column(Integer, nullable=False, default=25)
    w_education: Mapped[int] = mapped_column(Integer, nullable=False, default=15)
    w_semantic: Mapped[int] = mapped_column(Integer, nullable=False, default=20)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, nullable=False)

    min_level: Mapped[EducationLevel] = relationship(lazy="joined")
    skills: Mapped[list["JobSkill"]] = relationship(
        back_populates="job", cascade="all, delete-orphan", passive_deletes=True
    )

    __table_args__ = (
        CheckConstraint("length(trim(job_title)) BETWEEN 2 AND 100", name="ck_job_title"),
        CheckConstraint("length(trim(location)) BETWEEN 2 AND 80", name="ck_job_location"),
        CheckConstraint("status IN ('open','closed')", name="ck_job_status"),
        CheckConstraint("min_experience BETWEEN 0 AND 50", name="ck_job_min_exp"),
        CheckConstraint("length(job_description) BETWEEN 20 AND 5000", name="ck_job_desc"),
        CheckConstraint("w_skills BETWEEN 0 AND 100", name="ck_w_skills"),
        CheckConstraint("w_experience BETWEEN 0 AND 100", name="ck_w_experience"),
        CheckConstraint("w_education BETWEEN 0 AND 100", name="ck_w_education"),
        CheckConstraint("w_semantic BETWEEN 0 AND 100", name="ck_w_semantic"),
        CheckConstraint("w_skills + w_experience + w_education + w_semantic = 100", name="ck_w_total"),
    )


class JobSkill(Base):
    __tablename__ = "job_skill"
    job_id: Mapped[int] = mapped_column(
        ForeignKey("job_specification.job_id", ondelete="CASCADE"), primary_key=True
    )
    skill_id: Mapped[int] = mapped_column(ForeignKey("skill.skill_id", ondelete="CASCADE"), primary_key=True)
    skill_type: Mapped[str] = mapped_column(String(10), nullable=False)

    job: Mapped[JobSpecification] = relationship(back_populates="skills")
```

```python
    skill: Mapped[Skill] = relationship(lazy="joined")

    __table_args__ = (CheckConstraint("skill_type IN ('required','preferred')", name="ck_skill_type"),)


class MatchResult(Base):
    __tablename__ = "match_result"
    result_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    candidate_id: Mapped[int] = mapped_column(
        ForeignKey("candidate.candidate_id", ondelete="CASCADE"), nullable=False
    )
    job_id: Mapped[int] = mapped_column(
        ForeignKey("job_specification.job_id", ondelete="CASCADE"), nullable=False
    )
    skills_score: Mapped[float] = mapped_column(Float, nullable=False)
    experience_score: Mapped[float] = mapped_column(Float, nullable=False)
    education_score: Mapped[float] = mapped_column(Float, nullable=False)
    semantic_score: Mapped[float] = mapped_column(Float, nullable=False)
    overall_score: Mapped[float] = mapped_column(Float, nullable=False)
    rank: Mapped[int] = mapped_column(Integer, nullable=False)
    # {"required": [...], "preferred": [...], "aliases": {"js": "javascript"}}
    matched_skills: Mapped[dict] = mapped_column(JSON, nullable=False)
    # {"required": [...], "preferred": [...]}
    missing_skills: Mapped[dict] = mapped_column(JSON, nullable=False)
    run_at: Mapped[datetime] = mapped_column(DateTime, nullable=False)
    # Snapshot of the plain-English explanation and keyword-search notes at run
    # time, so they always agree with the stored scores.
    explanation: Mapped[str | None] = mapped_column(Text, nullable=True)
    insights: Mapped[list | None] = mapped_column(JSON, nullable=True)

    candidate: Mapped[Candidate] = relationship(lazy="joined")

    __table_args__ = (
        CheckConstraint("skills_score BETWEEN 0 AND 100", name="ck_mr_skills"),
        CheckConstraint("experience_score BETWEEN 0 AND 100", name="ck_mr_exp"),
        CheckConstraint("education_score BETWEEN 0 AND 100", name="ck_mr_edu"),
        CheckConstraint("semantic_score BETWEEN 0 AND 100", name="ck_mr_sem"),
        CheckConstraint("overall_score BETWEEN 0 AND 100", name="ck_mr_overall"),
        CheckConstraint("rank >= 1", name="ck_mr_rank"),
        UniqueConstraint("candidate_id", "job_id", "run_at", name="uq_mr_cand_job_run"),
        Index("ix_match_result_job_run", "job_id", "run_at"),
    )


class MatchRun(Base):
    """Metadata for one matching run (an addition to the brief's schema).

    Stores the weights actually used and how long the run took, so the Results
    screen stays truthful even if the job specification is edited afterwards.
    """

    __tablename__ = "match_run"
    run_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    job_id: Mapped[int] = mapped_column(
        ForeignKey("job_specification.job_id", ondelete="CASCADE"), nullable=False
    )
    run_at: Mapped[datetime] = mapped_column(DateTime, nullable=False)
    duration_ms: Mapped[int] = mapped_column(Integer, nullable=False)
    candidate_count: Mapped[int] = mapped_column(Integer, nullable=False)
    w_skills: Mapped[int] = mapped_column(Integer, nullable=False)
    w_experience: Mapped[int] = mapped_column(Integer, nullable=False)
    w_education: Mapped[int] = mapped_column(Integer, nullable=False)
    w_semantic: Mapped[int] = mapped_column(Integer, nullable=False)

    __table_args__ = (
        UniqueConstraint("job_id", "run_at", name="uq_run_job_time"),
        CheckConstraint("w_skills + w_experience + w_education + w_semantic = 100", name="ck_run_w_total"),
    )


# ---------------------------------------------------------------------------
# Phase 5 additions: user accounts, applicant pipeline, resource allocation
# ---------------------------------------------------------------------------
ROLES = ("admin", "recruiter", "resource_manager")
STAGES = ("applied", "screened", "interview", "offer", "hired", "rejected")
PROJECT_STATUSES = ("planned", "active", "on_hold", "completed")


class User(Base):
    """A person who logs in. The role decides which screens and actions they get."""

    __tablename__ = "app_user"
    user_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    full_name: Mapped[str] = mapped_column(String(100), nullable=False)
    email: Mapped[str] = mapped_column(String(254), unique=True, nullable=False)
    password_hash: Mapped[str] = mapped_column(String(100), nullable=False)
    role: Mapped[str] = mapped_column(String(20), nullable=False)
    is_active: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, nullable=False)
    last_login_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)

    __table_args__ = (
        CheckConstraint("role IN ('admin','recruiter','resource_manager')", name="ck_user_role"),
        CheckConstraint("email = lower(trim(email))", name="ck_user_email_lower"),
    )


class Application(Base):
    """One candidate applying for one job, and where that application stands."""

    __tablename__ = "application"
    application_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    candidate_id: Mapped[int] = mapped_column(
        ForeignKey("candidate.candidate_id", ondelete="CASCADE"), nullable=False
    )
    job_id: Mapped[int] = mapped_column(
```

```
        ForeignKey("job_specification.job_id", ondelete="CASCADE"), nullable=False
    )
    stage: Mapped[str] = mapped_column(String(12), nullable=False, default="applied")
    # Overall match score when the candidate was added from a match run.
    match_score: Mapped[float | None] = mapped_column(Float, nullable=True)
    applied_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, nullable=False)

    candidate: Mapped[Candidate] = relationship(lazy="joined")
    job: Mapped[JobSpecification] = relationship(lazy="joined")
    events: Mapped[list["ApplicationEvent"]] = relationship(
        back_populates="application",
        cascade="all, delete-orphan",
        passive_deletes=True,
        order_by="ApplicationEvent.event_id",
    )

    __table_args__ = (
        CheckConstraint(
            "stage IN ('applied','screened','interview','offer','hired','rejected')", name="ck_app_stage"
        ),
        CheckConstraint("match_score IS NULL OR match_score BETWEEN 0 AND 100", name="ck_app_score"),
        UniqueConstraint("candidate_id", "job_id", name="uq_app_cand_job"),
        Index("ix_application_job_stage", "job_id", "stage"),
    )


class ApplicationEvent(Base):
    """Audit trail: every stage change, who made it and when. Rows are never edited."""

    __tablename__ = "application_event"
    event_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    application_id: Mapped[int] = mapped_column(
        ForeignKey("application.application_id", ondelete="CASCADE"), nullable=False
    )
    from_stage: Mapped[str | None] = mapped_column(String(12), nullable=True)
    to_stage: Mapped[str] = mapped_column(String(12), nullable=False)
    changed_by: Mapped[int | None] = mapped_column(
        ForeignKey("app_user.user_id", ondelete="SET NULL"), nullable=True
    )
    changed_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, nullable=False)
    note: Mapped[str | None] = mapped_column(String(500), nullable=True)

    application: Mapped[Application] = relationship(back_populates="events")
    user: Mapped[User | None] = relationship(lazy="joined")


class StaffMember(Base):
    """Someone who can be allocated to projects: existing staff or a hired candidate."""

    __tablename__ = "staff_member"
    staff_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    full_name: Mapped[str] = mapped_column(String(100), nullable=False)
    email: Mapped[str | None] = mapped_column(String(254), nullable=True)
    role_title: Mapped[str] = mapped_column(String(100), nullable=False)
    location: Mapped[str | None] = mapped_column(String(80), nullable=True)
    # Set when the person was hired through the pipeline.
    candidate_id: Mapped[int | None] = mapped_column(
        ForeignKey("candidate.candidate_id", ondelete="SET NULL"), nullable=True, unique=True
    )
    source: Mapped[str] = mapped_column(String(10), nullable=False, default="existing")
    is_active: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, nullable=False)

    allocations: Mapped[list["Allocation"]] = relationship(
        back_populates="staff", cascade="all, delete-orphan", passive_deletes=True
    )

    __table_args__ = (
        CheckConstraint("length(trim(full_name)) BETWEEN 2 AND 100", name="ck_staff_name"),
        CheckConstraint("source IN ('existing','hired')", name="ck_staff_source"),
    )


class Project(Base):
    __tablename__ = "project"
    project_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    name: Mapped[str] = mapped_column(String(100), nullable=False, unique=True)
    client: Mapped[str] = mapped_column(String(100), nullable=False)
    description: Mapped[str | None] = mapped_column(Text, nullable=True)
    start_date: Mapped[date] = mapped_column(Date, nullable=False)
    end_date: Mapped[date] = mapped_column(Date, nullable=False)
    status: Mapped[str] = mapped_column(String(10), nullable=False, default="planned")
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, nullable=False)

    allocations: Mapped[list["Allocation"]] = relationship(
        back_populates="project", cascade="all, delete-orphan", passive_deletes=True
    )

    __table_args__ = (
        CheckConstraint("length(trim(name)) BETWEEN 2 AND 100", name="ck_project_name"),
        CheckConstraint("end_date >= start_date", name="ck_project_dates"),
        CheckConstraint("status IN ('planned','active','on_hold','completed')", name="ck_project_status"),
    )


class Allocation(Base):
    """A share of one staff member's time on one project for a date range."""

    __tablename__ = "allocation"
    allocation_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    staff_id: Mapped[int] = mapped_column(ForeignKey("staff_member.staff_id", ondelete="CASCADE"), nullable=False)
    project_id: Mapped[int] = mapped_column(ForeignKey("project.project_id", ondelete="CASCADE"), nullable=False)
    role_on_project: Mapped[str | None] = mapped_column(String(100), nullable=True)
    allocation_percent: Mapped[int] = mapped_column(Integer, nullable=False)
    start_date: Mapped[date] = mapped_column(Date, nullable=False)
```

```
    end_date: Mapped[date] = mapped_column(Date, nullable=False)

    staff: Mapped[StaffMember] = relationship(back_populates="allocations", lazy="joined")
    project: Mapped[Project] = relationship(back_populates="allocations", lazy="joined")

    __table_args__ = (
        CheckConstraint("allocation_percent BETWEEN 1 AND 100", name="ck_alloc_percent"),
        CheckConstraint("end_date >= start_date", name="ck_alloc_dates"),
        Index("ix_allocation_staff", "staff_id"),
    )
```

backend/app/security.py

Password hashing, tokens, role checks, lock-out.

```
"""Authentication and role-based access control.

- Passwords are stored only as bcrypt hashes (passlib).
- After login the browser receives a signed JWT (HS256) that it sends back in
  the Authorization header. The token carries the user id and role and expires
  after TOKEN_HOURS hours.
- Every protected route declares the roles it allows with require_roles(...).
- Destructive admin actions (reset data, restore a backup) also need the
  admin's password again in the X-Confirm-Password header.
"""
from __future__ import annotations

import logging
import os
import secrets
import threading
import time
from datetime import datetime, timedelta, timezone

import jwt
from fastapi import Depends, Header
from passlib.context import CryptContext
from sqlalchemy.orm import Session

from .config import settings
from .database import get_db
from .errors import AppError, Forbidden, Unauthorized

log = logging.getLogger("hiretrack.security")
# 12 rounds in normal use; tests lower it (HIRETRACK_BCRYPT_ROUNDS=4) to stay fast.
pwd_context = CryptContext(
    schemes=["bcrypt"], deprecated="auto", bcrypt__rounds=int(os.getenv("HIRETRACK_BCRYPT_ROUNDS", "12"))
)

TOKEN_HOURS = 8
ALGORITHM = "HS256"

ROLE_LABELS = {"admin": "Administrator", "recruiter": "Recruiter (HR)", "resource_manager": "Resource Manager"}

# Shorthand role groups used by the routers.
ANY_ROLE = ("admin", "recruiter", "resource_manager")
HIRING = ("admin", "recruiter")
RESOURCING = ("admin", "resource_manager")
ADMIN = ("admin",)


# ---------------------------------------------------------------------- passwords
def hash_password(password: str) -> str:
    return pwd_context.hash(password)


def verify_password(password: str | None, password_hash: str | None) -> bool:
    if not password or not password_hash:
        return False
    try:
        return pwd_context.verify(password, password_hash)
    except (ValueError, TypeError):
        return False


# ---------------------------------------------------------------------- JWT secret
_secret_lock = threading.Lock()
_secret: str | None = None


def jwt_secret() -> str:
    """JWT_SECRET from .env, otherwise a random secret kept in data/.jwt_secret.

    Generating it on first run means a fresh install needs no manual step, and
    keeping it in a file means logins survive a restart.
    """
    global _secret
    if settings.jwt_secret:
        return settings.jwt_secret
    with _secret_lock:
        if _secret:
            return _secret
        path = settings.data_dir / ".jwt_secret"
        if path.exists():
            _secret = path.read_text(encoding="utf-8").strip()
        if not _secret:
            _secret = secrets.token_urlsafe(48)
            path.write_text(_secret, encoding="utf-8")
        return _secret


```

```python
def create_token(user_id: int, role: str, hours: float = TOKEN_HOURS) -> str:
    now = datetime.now(timezone.utc)
    payload = {"sub": str(user_id), "role": role, "iat": now, "exp": now + timedelta(hours=hours)}
    return jwt.encode(payload, jwt_secret(), algorithm=ALGORITHM)


def decode_token(token: str) -> dict:
    try:
        return jwt.decode(token, jwt_secret(), algorithms=[ALGORITHM])
    except jwt.ExpiredSignatureError as exc:
        raise Unauthorized("Your session has expired. Please sign in again.") from exc
    except jwt.InvalidTokenError as exc:
        raise Unauthorized("Please sign in to continue.") from exc


# ------------------------------------------------------------ login rate limiting
# Simple in-memory lockout: 5 failed attempts for one email locks it for 60 s.
MAX_FAILS = 5
LOCK_SECONDS = 60
_fails: dict[str, list[float]] = {}
_fails_lock = threading.Lock()


def check_not_locked(email: str) -> None:
    with _fails_lock:
        recent = [t for t in _fails.get(email, []) if time.monotonic() - t < LOCK_SECONDS]
        _fails[email] = recent
        if len(recent) >= MAX_FAILS:
            wait = int(LOCK_SECONDS - (time.monotonic() - recent[0])) + 1
            raise AppError(
                f"Too many failed sign-in attempts. Try again in {wait} seconds.", field="password", status_code=429
            )


def record_failure(email: str) -> None:
    with _fails_lock:
        _fails.setdefault(email, []).append(time.monotonic())


def clear_failures(email: str) -> None:
    with _fails_lock:
        _fails.pop(email, None)


# ------------------------------------------------------------ dependencies
def current_user(authorization: str | None = Header(default=None), db: Session = Depends(get_db)):
    """FastAPI dependency: the logged-in, active user (or 401)."""
    from .models import User

    if not authorization or not authorization.lower().startswith("bearer "):
        raise Unauthorized("Please sign in to continue.")
    payload = decode_token(authorization.split(" ", 1)[1].strip())
    try:
        user_id = int(payload.get("sub", ""))
    except ValueError as exc:
        raise Unauthorized("Please sign in to continue.") from exc
    user = db.get(User, user_id)
    if user is None or not user.is_active:
        raise Unauthorized("This account is no longer active.")
    return user


def require_roles(*roles: str):
    """Dependency factory: allow only users whose role is in `roles` (else 403)."""

    def _check(user=Depends(current_user)):
        if user.role not in roles:
            log.warning("Denied user id=%s role=%s (needs %s)", user.user_id, user.role, "/".join(roles))
            raise Forbidden("Your role does not have access to this action.")
        return user

    return _check


def require_confirmed_admin(
    x_confirm_password: str | None = Header(default=None),
    user=Depends(require_roles(*ADMIN)),
):
    """Admin AND the admin's own password re-entered (for destructive actions)."""
    if not verify_password(x_confirm_password, user.password_hash):
        log.warning("Rejected destructive admin action: password not confirmed (user id=%s)", user.user_id)
        raise Unauthorized("Incorrect password", field="password")
    return user
```

backend/app/engine/scoring.py

Four scores and the overall score.

```python
"""HireTrack matching engine: pure scoring functions.

This module has NO database or web code, so every rule can be unit tested on
its own. The service layer turns ORM rows into the plain dataclasses below,
calls `score_candidates`, and saves the result.

How a candidate is scored against a job (each component is 0-100):

  Skills      (required matched / required) x 80 + (preferred matched / preferred) x 20
              If the job has no preferred skills, required skills count for the full 100.
  Experience  100 if years >= minimum (or minimum is 0), else years / minimum x 100
  Education   100 if candidate level >= minimum level, else level / minimum x 100
  Semantic    cosine similarity of CV summary vs job description embeddings,
```

```
                negatives clipped to 0, x 100
  Overall       (skills x w_skills + experience x w_experience
                 + education x w_education + semantic x w_semantic) / 100

Every component is rounded to 1 decimal place (half up) *before* the overall
score is calculated, so the calculation shown on screen reproduces exactly.

Ranking: overall descending, then skills descending, then name A-Z.

FAIRNESS: scoring only ever reads skills, years of experience, education level
and the CV summary text. Name, email, age, gender and race are never inputs to
any score. The name is used only as the final alphabetical tie-break so that
the order is stable and predictable.
"""
from __future__ import annotations

from collections.abc import Callable, Sequence
from dataclasses import dataclass, field
from decimal import ROUND_HALF_UP, Decimal

import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# ---------------------------------------------------------------------------
# Constants
# ---------------------------------------------------------------------------
REQUIRED_SHARE = 80  # points available for required skills when preferred exist
PREFERRED_SHARE = 20  # points available for preferred skills
DEFAULT_WEIGHTS = {"skills": 40, "experience": 25, "education": 15, "semantic": 20}

EDUCATION_NAMES = {
    1: "Matric",
    2: "Higher Certificate",
    3: "Diploma",
    4: "Bachelor's Degree",
    5: "Honours",
    6: "Master's",
    7: "Doctorate",
}

# Thresholds for the plain-English explanation and the keyword-miss callout.
# all-MiniLM-L6-v2 cosine similarities for genuinely related CV/job pairs sit
# roughly between 0.45 and 0.75, so the bands are set on that scale.
SEMANTIC_CLOSE = 55.0
SEMANTIC_RELATED = 40.0
KEYWORD_MISS_SEMANTIC_MIN = 60.0  # "high" semantic score
KEYWORD_MISS_SKILLS_MAX = 60.0  # "low" skills score


# ---------------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------------
def round1(value: float) -> float:
    """Round to 1 decimal place, half up (2.25 -> 2.3), unlike Python's banker's round."""
    return float(Decimal(str(value)).quantize(Decimal("0.1"), rounding=ROUND_HALF_UP))


def clamp(value: float, low: float = 0.0, high: float = 100.0) -> float:
    return max(low, min(high, value))


def fmt_num(value: float) -> str:
    """90.0 -> '90', 62.3 -> '62.3' (for readable formulas)."""
    return f"{value:.1f}".rstrip("0").rstrip(".")


# ---------------------------------------------------------------------------
# Inputs / outputs
# ---------------------------------------------------------------------------
class WeightError(ValueError):
    """Raised when scoring weights are out of range or do not total 100."""


@dataclass(frozen=True)
class Weights:
    skills: int = DEFAULT_WEIGHTS["skills"]
    experience: int = DEFAULT_WEIGHTS["experience"]
    education: int = DEFAULT_WEIGHTS["education"]
    semantic: int = DEFAULT_WEIGHTS["semantic"]

    @property
    def total(self) -> int:
        return self.skills + self.experience + self.education + self.semantic

    def validate(self) -> "Weights":
        for name in ("skills", "experience", "education", "semantic"):
            v = getattr(self, name)
            if not isinstance(v, int) or isinstance(v, bool) or not 0 <= v <= 100:
                raise WeightError(f"Each weight must be a whole number from 0 to 100 ({name} is {v})")
        if self.total != 100:
            raise WeightError(f"Weights must add up to 100% (now {self.total}%)")
        return self


@dataclass
class JobCriteria:
    required: list[str]  # canonical skill names
    preferred: list[str]
    min_experience: float
    min_level: int
    description: str
    weights: Weights = field(default_factory=Weights)


@dataclass
class CandidateProfile:
```

```
    candidate_id: int
    skills: list[str]  # canonical skill names
    years_experience: float
    level: int
    resume_summary: str
    # Only used for the final alphabetical tie-break, never for scoring.
    full_name: str = ""
    # canonical -> what was originally typed, e.g. {"javascript": "JS"}
    skills_as_entered: dict[str, str] = field(default_factory=dict)


@dataclass
class SkillMatch:
    score: float
    matched_required: list[str]
    missing_required: list[str]
    matched_preferred: list[str]
    missing_preferred: list[str]


@dataclass
class CandidateScore:
    candidate_id: int
    full_name: str
    skills: float
    experience: float
    education: float
    semantic: float
    overall: float
    skill_match: SkillMatch
    alias_hits: dict[str, str]  # canonical -> as entered, for matched skills only
    explanation: str = ""
    keyword_insights: list[str] = field(default_factory=list)
    rank: int = 0


# ---------------------------------------------------------------------------
# Validation rules shared with the API layer
# ---------------------------------------------------------------------------
def validate_skill_lists(required: Sequence[str], preferred: Sequence[str]) -> None:
    """A job needs >= 1 required skill and preferred must not repeat a required one."""
    if not required:
        raise ValueError("Add at least one required skill")
    overlap = sorted(set(required) & set(preferred))
    if overlap:
        raise ValueError(f"Preferred skills repeat required skills: {', '.join(overlap)}")


# ---------------------------------------------------------------------------
# Component scores
# ---------------------------------------------------------------------------
def skills_score(candidate_skills: Sequence[str], required: Sequence[str], preferred: Sequence[str]) -> SkillMatch:
    """Required skills are worth 80 points, preferred 20.

    With no preferred skills, required skills are worth the full 100.
    Inputs must already be canonical (see SkillNormaliser).
    """
    have = set(candidate_skills)
    matched_req = [s for s in required if s in have]
    missing_req = [s for s in required if s not in have]
    matched_pref = [s for s in preferred if s in have]
    missing_pref = [s for s in preferred if s not in have]

    if not required:
        req_part = 0.0
    elif preferred:
        req_part = len(matched_req) / len(required) * REQUIRED_SHARE
    else:
        req_part = len(matched_req) / len(required) * 100
    pref_part = len(matched_pref) / len(preferred) * PREFERRED_SHARE if preferred else 0.0

    return SkillMatch(
        score=round1(clamp(req_part + pref_part)),
        matched_required=matched_req,
        missing_required=missing_req,
        matched_preferred=matched_pref,
        missing_preferred=missing_pref,
    )


def experience_score(years: float, minimum: float) -> float:
    """100 if the candidate meets the minimum (or there is none), else pro rata."""
    if minimum <= 0 or years >= minimum:
        return 100.0
    return round1(clamp(max(years, 0) / minimum * 100))


def education_score(level: int, min_level: int) -> float:
    """100 if the candidate's level is at or above the minimum, else level / minimum x 100."""
    if level >= min_level:
        return 100.0
    return round1(clamp(level / min_level * 100))


def semantic_scores(job_vector: np.ndarray, candidate_matrix: np.ndarray) -> np.ndarray:
    """Cosine similarity of each CV embedding to the job embedding, as 0-100.

    Negative similarities (opposite meaning) are treated as 0.
    """
    if candidate_matrix.shape[0] == 0:
        return np.zeros(0)
    sims = cosine_similarity(job_vector.reshape(1, -1), candidate_matrix)[0]
    return np.clip(sims, 0.0, None) * 100.0


def overall_score(skills: float, experience: float, education: float, semantic: float, weights: Weights) -> float:
```

```
    """Weighted average of the four components, rounded to 1 decimal.

    Computed in integer tenths so the browser's "what if" re-ranking (which
    uses the same integer method) always agrees with the server exactly.
    """
    tenths = [int(round(c * 10)) for c in (skills, experience, education, semantic)]
    ws = (weights.skills, weights.experience, weights.education, weights.semantic)
    total = sum(t * w for t, w in zip(tenths, ws))  # in (tenths x percent)
    return ((total + 50) // 100) / 10  # round half up back to tenths


def calculation_string(score: CandidateScore, weights: Weights) -> str:
    """Human-readable formula, e.g. 'Overall = (90×40 + 100×25 + 80×15 + 62.3×20) / 100 = 86.5'."""
    parts = [
        f"{fmt_num(score.skills)}×{weights.skills}",
        f"{fmt_num(score.experience)}×{weights.experience}",
        f"{fmt_num(score.education)}×{weights.education}",
        f"{fmt_num(score.semantic)}×{weights.semantic}",
    ]
    return f"Overall = ({' + '.join(parts)}) / 100 = {score.overall:.1f}"


# ---------------------------------------------------------------------------
# Plain-English explanation
# ---------------------------------------------------------------------------
def _years(n: float) -> str:
    n_txt = fmt_num(n)
    return f"{n_txt} year" if n_txt == "1" else f"{n_txt} years"


def explain(
    skill_match: SkillMatch,
    years: float,
    min_experience: float,
    level: int,
    min_level: int,
    semantic: float,
) -> str:
    """One jargon-free sentence describing why a candidate scored as they did."""
    n_req = len(skill_match.matched_required) + len(skill_match.missing_required)
    got = len(skill_match.matched_required)
    n_pref = len(skill_match.matched_preferred) + len(skill_match.missing_preferred)

    if n_req and got == n_req:
        skills = f"strong on skills ({got} of {n_req} required"
    elif n_req and got / n_req >= 0.5:
        skills = f"partial skills match ({got} of {n_req} required"
    else:
        skills = f"weak skills match ({got} of {n_req} required"
    if n_pref:
        skills += f", {len(skill_match.matched_preferred)} of {n_pref} preferred"
    skills += ")"

    if min_experience <= 0:
        exp = "no experience minimum applies"
    elif years >= min_experience:
        exp = "meets the experience minimum"
    elif years <= 0:
        exp = f"no experience yet against the {fmt_num(min_experience)}-year minimum"
    else:
        exp = f"{_years(min_experience - years)} short of the {fmt_num(min_experience)}-year minimum"

    gap = min_level - level
    if gap <= 0:
        edu = "meets the education requirement"
    elif gap == 1:
        edu = "one level below the education requirement"
    else:
        edu = f"{gap} levels below the education requirement"

    if semantic >= SEMANTIC_CLOSE:
        sem = "CV closely matches the role description"
    elif semantic >= SEMANTIC_RELATED:
        sem = "CV is broadly related to the role"
    else:
        sem = "CV describes different work"

    sentence = f"{skills}, {exp}, {edu}; {sem}."
    return sentence[0].upper() + sentence[1:]


def keyword_insights(skill_match: SkillMatch, semantic: float, skills: float, alias_hits: dict[str, str]) -> list[str]:
    """Reasons a plain keyword search would have missed or under-rated this candidate."""
    notes: list[str] = []
    for canonical, entered in alias_hits.items():
        notes.append(f'Listed "{entered}", which a keyword search for "{canonical}" would not find.')
    if semantic >= KEYWORD_MISS_SEMANTIC_MIN and skills < KEYWORD_MISS_SKILLS_MAX:
        n_req = len(skill_match.matched_required) + len(skill_match.missing_required)
        notes.append(
            f"Their CV describes this kind of work in different words: meaning match is "
            f"{fmt_num(semantic)} while only {len(skill_match.matched_required)} of {n_req} "
            f"required skills match by name."
        )
    return notes


# ---------------------------------------------------------------------------
# Whole run
# ---------------------------------------------------------------------------
def score_candidate(job: JobCriteria, cand: CandidateProfile, semantic_raw: float) -> CandidateScore:
    """Score one candidate given its pre-computed semantic similarity (0-100)."""
    sm = skills_score(cand.skills, job.required, job.preferred)
    exp = experience_score(cand.years_experience, job.min_experience)
    edu = education_score(cand.level, job.min_level)
    sem = round1(clamp(semantic_raw))
    overall = overall_score(sm.score, exp, edu, sem, job.weights)
```

```
    matched = set(sm.matched_required) | set(sm.matched_preferred)
    alias_hits = {
        c: entered
        for c, entered in cand.skills_as_entered.items()
        if c in matched and entered and entered.strip().lower() != c
    }
    return CandidateScore(
        candidate_id=cand.candidate_id,
        full_name=cand.full_name,
        skills=sm.score,
        experience=exp,
        education=edu,
        semantic=sem,
        overall=overall,
        skill_match=sm,
        alias_hits=alias_hits,
        explanation=explain(sm, cand.years_experience, job.min_experience, cand.level, job.min_level, sem),
        keyword_insights=keyword_insights(sm, sem, sm.score, alias_hits),
    )


def rank_scores(scores: list[CandidateScore]) -> list[CandidateScore]:
    """Sort by overall desc, skills desc, name A-Z (then id, for full determinism) and set rank."""
    ordered = sorted(scores, key=lambda s: (-s.overall, -s.skills, s.full_name.casefold(), s.candidate_id))
    for i, s in enumerate(ordered, start=1):
        s.rank = i
    return ordered


def score_candidates(
    job: JobCriteria,
    candidates: Sequence[CandidateProfile],
    embed: Callable[[Sequence[str]], np.ndarray],
) -> list[CandidateScore]:
    """Score and rank every candidate for one job.

    `embed` maps a list of texts to an (n, dim) matrix. The job description is
    encoded once; candidate summaries are encoded in one batch (and cached by
    the embedder between runs).
    """
    job.weights.validate()
    validate_skill_lists(job.required, job.preferred)
    if not candidates:
        return []
    job_vec = embed([job.description])[0]
    cand_mat = embed([c.resume_summary for c in candidates])
    sems = semantic_scores(job_vec, cand_mat)
    return rank_scores([score_candidate(job, c, float(s)) for c, s in zip(candidates, sems)])
```

backend/app/engine/skills.py

Skill normalisation and aliases.

```
"""Skill normalisation.

Recruiters and candidates write the same skill many ways ("JS", "javascript",
"Java Script "). Everything is reduced to one canonical, lowercase form before
it is stored or compared, so "JS" on a CV matches "JavaScript" on a job.
"""
from __future__ import annotations

import re
from collections.abc import Iterable

# alias -> canonical. Keys and values must already be lowercase/space-collapsed.
ALIASES: dict[str, str] = {
    # Languages
    "js": "javascript",
    "java script": "javascript",
    "ecmascript": "javascript",
    "es6": "javascript",
    "ts": "typescript",
    "py": "python",
    "python3": "python",
    "python 3": "python",
    "golang": "go",
    "csharp": "c#",
    "c sharp": "c#",
    "cpp": "c++",
    "dotnet": ".net",
    "dot net": ".net",
    ".net core": ".net",
    "asp.net core": "asp.net",
    "vb": "visual basic",
    "vba": "excel vba",
    # Frameworks / runtimes
    "node": "node.js",
    "nodejs": "node.js",
    "node js": "node.js",
    "reactjs": "react",
    "react.js": "react",
    "react js": "react",
    "vuejs": "vue",
    "vue.js": "vue",
    "angularjs": "angular",
    "nextjs": "next.js",
    "next js": "next.js",
    "expressjs": "express",
    "express.js": "express",
    "springboot": "spring boot",
```

```
    "sklearn": "scikit-learn",
    "scikit learn": "scikit-learn",
    # Data
    "postgres": "postgresql",
    "psql": "postgresql",
    "postgre sql": "postgresql",
    "mssql": "sql server",
    "ms sql": "sql server",
    "microsoft sql server": "sql server",
    "t-sql": "tsql",
    "mongo": "mongodb",
    "elastic": "elasticsearch",
    "apache spark": "spark",
    "pyspark": "spark",
    "apache kafka": "kafka",
    "apache airflow": "airflow",
    "dbt core": "dbt",
    "ms excel": "excel",
    "microsoft excel": "excel",
    "powerbi": "power bi",
    "pbi": "power bi",
    "ms power bi": "power bi",
    "etl/elt": "etl",
    "data warehouse": "data warehousing",
    "dwh": "data warehousing",
    # AI / ML
    "ml": "machine learning",
    "dl": "deep learning",
    "ai": "artificial intelligence",
    "nlp": "natural language processing",
    "genai": "generative ai",
    "llm": "large language models",
    "llms": "large language models",
    # Cloud / DevOps
    "k8s": "kubernetes",
    "kube": "kubernetes",
    "amazon web services": "aws",
    "microsoft azure": "azure",
    "ms azure": "azure",
    "google cloud": "gcp",
    "google cloud platform": "gcp",
    "cicd": "ci/cd",
    "ci cd": "ci/cd",
    "gh actions": "github actions",
    "iac": "infrastructure as code",
    "docker compose": "docker",
    "linux admin": "linux",
    # Practices
    "rest": "rest apis",
    "rest api": "rest apis",
    "restful": "rest apis",
    "restful apis": "rest apis",
    "oop": "object-oriented programming",
    "tdd": "test-driven development",
    "ux": "ux design",
    "ui/ux": "ux design",
    "pm": "project management",
    "ba": "business analysis",
    "bpmn 2.0": "bpmn",
    "uml diagrams": "uml",
    "pen testing": "penetration testing",
    "pentesting": "penetration testing",
    "siem tools": "siem",
    "itil v4": "itil",
    "itil4": "itil",
}

_SPACE_RE = re.compile(r"\s+")


class SkillNormaliser:
    """Turns free-text skill names into canonical lowercase names."""

    def __init__(self, aliases: dict[str, str] | None = None) -> None:
        self.aliases = dict(ALIASES if aliases is None else aliases)

    @staticmethod
    def clean(raw: str) -> str:
        """Lowercase, trim and collapse internal whitespace (no alias lookup)."""
        return _SPACE_RE.sub(" ", (raw or "").strip().lower())

    def normalise(self, raw: str) -> str:
        """Canonical form of one skill. Returns "" for blank input."""
        cleaned = self.clean(raw).strip(" ,;")
        return self.aliases.get(cleaned, cleaned)

    def is_alias(self, raw: str) -> bool:
        """True when the raw text was rewritten by the alias table (e.g. JS)."""
        cleaned = self.clean(raw)
        return cleaned in self.aliases and self.aliases[cleaned] != cleaned

    def normalise_many(self, raws: Iterable[str]) -> list[str]:
        """Normalise a list, dropping blanks and duplicates, keeping first-seen order."""
        seen: set[str] = set()
        out: list[str] = []
        for raw in raws:
            name = self.normalise(raw)
            if name and name not in seen:
                seen.add(name)
                out.append(name)
        return out

    def normalise_with_source(self, raws: Iterable[str]) -> list[tuple[str, str]]:
        """Like normalise_many but keeps what the user originally typed.

        Returns [(canonical, as_entered)]. Used to show "JS was read as
```

```
        JavaScript" in the results explanation.
        """
        seen: set[str] = set()
        out: list[tuple[str, str]] = []
        for raw in raws:
            name = self.normalise(raw)
            if name and name not in seen:
                seen.add(name)
                out.append((name, (raw or "").strip()))
        return out


normaliser = SkillNormaliser()
```

backend/app/engine/embeddings.py

Language-model embeddings and cache.

```
"""Sentence embeddings for the semantic score.

Loads all-MiniLM-L6-v2 from the local models/ folder (CPU only, offline) and
caches every embedding in memory, keyed by the exact text. Re-running a match
or matching 2,000 candidates against a second job only encodes text that has
never been seen before, in batches.

If the model is missing (not yet downloaded) or disabled for tests, a small
deterministic bag-of-words hashing embedder is used so the app keeps working;
a warning is logged and /api/stats reports which embedder is active.
"""
from __future__ import annotations

import hashlib
import logging
import re
import threading
from collections.abc import Sequence

import numpy as np

from ..config import settings

log = logging.getLogger("hiretrack.embeddings")

_WORD_RE = re.compile(r"[a-z0-9+#.]+")


class HashingEmbedder:
    """Fallback: signed feature hashing of word unigrams + bigrams, L2-normalised."""

    name = "hashing-fallback"
    dim = 384

    def encode(self, texts: Sequence[str]) -> np.ndarray:
        out = np.zeros((len(texts), self.dim), dtype=np.float32)
        for i, text in enumerate(texts):
            words = _WORD_RE.findall(text.lower())
            grams = words + [f"{a} {b}" for a, b in zip(words, words[1:])]
            for g in grams:
                h = int.from_bytes(hashlib.md5(g.encode()).digest()[:4], "little")
                out[i, h % self.dim] += 1.0 if (h >> 31) & 1 else -1.0
            norm = np.linalg.norm(out[i])
            if norm:
                out[i] /= norm
        return out


class Embedder:
    """Thread-safe, cached text -> vector encoder.

    The cache is keyed by the SHA-1 of the text (so no CV text is held twice or
    written to disk) and persisted to data/embedding_cache.npz, so CV summaries
    are only ever encoded once, even across restarts.
    """

    def __init__(self) -> None:
        self._model = None
        self._backend_name = "not-loaded"
        self._cache: dict[str, np.ndarray] = {}
        self._lock = threading.Lock()
        self._dirty = False

    @staticmethod
    def _key(text: str) -> str:
        return hashlib.sha1(text.encode("utf-8")).hexdigest()

    @property
    def _cache_file(self):
        return settings.data_dir / "embedding_cache.npz"

    # -- model loading ------------------------------------------------------
    def _load(self) -> None:
        if self._model is not None:
            return
        if settings.disable_model:
            self._model, self._backend_name = HashingEmbedder(), HashingEmbedder.name
            return
        local = settings.model_dir / "all-MiniLM-L6-v2"
        try:
            from sentence_transformers import SentenceTransformer

            if not (local / "model.safetensors").exists():
```

```
                raise FileNotFoundError(f"{local} not found. Run: python -m app.download_model")
            self._model = SentenceTransformer(str(local), device="cpu")
            self._backend_name = "all-MiniLM-L6-v2"
            log.info("Loaded sentence-transformers model from %s", local)
            self._load_disk_cache()
        except Exception as exc:  # pragma: no cover - depends on environment
            log.warning("Semantic model unavailable (%s); using hashing fallback", exc)
            self._model, self._backend_name = HashingEmbedder(), HashingEmbedder.name

    def _load_disk_cache(self) -> None:
        try:
            if self._cache_file.exists():
                with np.load(self._cache_file, allow_pickle=False) as z:
                    if str(z["model"]) != self._backend_name:
                        return
                    for k, v in zip(z["keys"], z["vectors"]):
                        self._cache[str(k)] = v
                log.info("Loaded %d cached embeddings from disk", len(self._cache))
        except Exception:  # pragma: no cover - a corrupt cache is simply rebuilt
            log.warning("Embedding cache unreadable; it will be rebuilt")
            self._cache.clear()

    def save(self) -> None:
        """Write the cache to disk if anything new was encoded (real model only)."""
        with self._lock:
            if not self._dirty or isinstance(self._model, HashingEmbedder) or not self._cache:
                return
            keys = np.array(list(self._cache.keys()))
            vectors = np.vstack(list(self._cache.values())).astype(np.float32)
            tmp = self._cache_file.with_suffix(".tmp.npz")
            np.savez(tmp, keys=keys, vectors=vectors, model=np.array(self._backend_name))
            tmp.replace(self._cache_file)
            self._dirty = False
            log.info("Saved %d embeddings to disk cache", len(keys))

    @property
    def backend_name(self) -> str:
        return self._backend_name

    def warm_up(self) -> None:
        with self._lock:
            self._load()

    # -- encoding ------------------------------------------------------------
    def encode(self, texts: Sequence[str], batch_size: int = 64) -> np.ndarray:
        """Return an (n, dim) matrix of L2-normalised embeddings for `texts`.

        Only texts missing from the cache are sent to the model, in batches.
        """
        with self._lock:
            self._load()
            keys = [self._key(t) for t in texts]
            missing = {}
            for k, t in zip(keys, texts):
                if k not in self._cache and k not in missing:
                    missing[k] = t
            if missing:
                todo = list(missing.values())
                if isinstance(self._model, HashingEmbedder):
                    vecs = self._model.encode(todo)
                else:
                    vecs = self._model.encode(
                        todo,
                        batch_size=batch_size,
                        convert_to_numpy=True,
                        normalize_embeddings=True,
                        show_progress_bar=False,
                    )
                for k, v in zip(missing.keys(), vecs):
                    self._cache[k] = np.asarray(v, dtype=np.float32)
                self._dirty = True
            if not texts:
                return np.zeros((0, 384), dtype=np.float32)
            return np.vstack([self._cache[k] for k in keys])

    def cached(self, texts: Sequence[str]) -> int:
        """How many of `texts` are already encoded (for the "preparing" status)."""
        return sum(1 for t in texts if self._key(t) in self._cache)

    def cache_size(self) -> int:
        return len(self._cache)

    def clear_cache(self) -> None:
        with self._lock:
            self._cache.clear()


embedder = Embedder()
```

backend/app/engine/pipeline.py

Pipeline stage rules.

```
"""Applicant pipeline rules (pure Python, no database code).

Stages run in a fixed order:

    applied -> screened -> interview -> offer -> hired

An application can move forward ONE stage at a time, or be rejected from any
stage that is still open. "hired" and "rejected" are final. A rejected
```

```python
application can be reopened, which puts it back at the stage it was rejected
from, so a mistake can be undone without losing the audit history.
"""
from __future__ import annotations

FLOW: tuple[str, ...] = ("applied", "screened", "interview", "offer", "hired")
REJECTED = "rejected"
ALL_STAGES: tuple[str, ...] = (*FLOW, REJECTED)
FINAL: frozenset[str] = frozenset({"hired", REJECTED})

LABELS = {
    "applied": "Applied",
    "screened": "Screened",
    "interview": "Interview",
    "offer": "Offer",
    "hired": "Hired",
    "rejected": "Rejected",
}


class StageError(ValueError):
    """Raised when a stage change is not allowed."""


def next_stage(stage: str) -> str | None:
    """The stage after this one in the normal flow, or None at the end."""
    if stage not in FLOW:
        return None
    i = FLOW.index(stage)
    return FLOW[i + 1] if i + 1 < len(FLOW) else None


def allowed_moves(stage: str, reopen_to: str | None = None) -> list[str]:
    """Stages an application at `stage` may move to.

    reopen_to is the stage a rejected application was rejected from.
    """
    if stage not in ALL_STAGES:
        raise StageError(f"Unknown stage '{stage}'")
    if stage == REJECTED:
        return [reopen_to] if reopen_to in FLOW and reopen_to != "hired" else []
    if stage == "hired":
        return []
    moves = [next_stage(stage)]
    moves.append(REJECTED)
    return [m for m in moves if m]


def check_move(current: str, target: str, reopen_to: str | None = None) -> None:
    """Raise StageError with a plain-English reason if current -> target is not allowed."""
    if target not in ALL_STAGES:
        raise StageError(f"Unknown stage '{target}'")
    if current == target:
        raise StageError(f"The application is already at {LABELS[current]}")
    if current in FINAL and not (current == REJECTED and target == reopen_to):
        raise StageError(f"{LABELS[current]} is a final stage and cannot be changed")
    if target not in allowed_moves(current, reopen_to):
        nxt = next_stage(current)
        hint = f" The next stage is {LABELS[nxt]}." if nxt else ""
        raise StageError(f"Cannot move from {LABELS[current]} to {LABELS[target]}.{hint}")


def furthest_stage(stage: str, rejected_from: str | None) -> str:
    """The furthest stage in the normal flow an application got to.

    A rejected application counts as having reached the stage it was rejected at.
    """
    if stage == REJECTED:
        return rejected_from if rejected_from in FLOW else "applied"
    return stage


def funnel(furthest: dict[str, int], current: dict[str, int]) -> list[dict]:
    """Build the funnel rows shown on the dashboard.

    furthest: {stage: number of applications whose furthest stage is this one}
    current:  {stage: number of applications sitting at this stage right now}
    "reached" = applications that got to this stage or any later one.
    "conversion" = reached / reached at the previous stage, as a percentage.
    """
    out = []
    prev = None
    for i, stage in enumerate(FLOW):
        reached = sum(furthest.get(s, 0) for s in FLOW[i:])
        conv = round(reached / prev * 100, 1) if prev else None
        out.append(
            {
                "stage": stage,
                "label": LABELS[stage],
                "current": current.get(stage, 0),
                "reached": reached,
                "conversion": conv,
            }
        )
        prev = reached
    out.append(
        {
            "stage": REJECTED,
            "label": LABELS[REJECTED],
            "current": current.get(REJECTED, 0),
            "reached": None,
            "conversion": None,
        }
    )
    return out
```

backend/app/engine/allocation.py

Allocation and utilisation rules.

```python
"""Resource allocation maths (pure Python, no database code).

A staff member's load on a given day is the sum of allocation_percent for every
allocation that covers that day. Nobody may be booked above 100% on any day.
Utilisation is the load on the reference day (normally today).
"""
from __future__ import annotations

from dataclasses import dataclass
from datetime import date, timedelta
from typing import Iterable

MAX_LOAD = 100


@dataclass(frozen=True)
class Booking:
    percent: int
    start: date
    end: date
    allocation_id: int | None = None


def overlaps(a_start: date, a_end: date, b_start: date, b_end: date) -> bool:
    return a_start <= b_end and b_start <= a_end


def load_on(bookings: Iterable[Booking], day: date) -> int:
    """Total percent booked on one day."""
    return sum(b.percent for b in bookings if b.start <= day <= b.end)


def peak_load(bookings: Iterable[Booking], start: date, end: date) -> tuple[int, date | None]:
    """Highest total load on any day in [start, end], and the first day it happens.

    Load only changes on a booking's start day or the day after its end, so it
    is enough to check those days (a sweep), not every day in the range.
    """
    inside = [b for b in bookings if overlaps(b.start, b.end, start, end)]
    if not inside:
        return 0, None
    days = {start}
    for b in inside:
        if start <= b.start <= end:
            days.add(b.start)
        after = b.end + timedelta(days=1)
        if start <= after <= end:
            days.add(after)
    best, best_day = 0, None
    for d in sorted(days):
        load = load_on(inside, d)
        if load > best:
            best, best_day = load, d
    return best, best_day


def check_new_booking(existing: Iterable[Booking], new: Booking, ignore_id: int | None = None) -> None:
    """Raise ValueError if adding `new` would put anyone above 100% on any day."""
    if new.end < new.start:
        raise ValueError("End date cannot be before the start date")
    if not 1 <= new.percent <= MAX_LOAD:
        raise ValueError("Allocation must be between 1% and 100%")
    others = [b for b in existing if ignore_id is None or b.allocation_id != ignore_id]
    combined = [*others, new]
    peak, day = peak_load(combined, new.start, new.end)
    if peak > MAX_LOAD:
        already = peak - new.percent
        raise ValueError(
            f"This would book the person at {peak}% on {day:%d %b %Y} "
            f"(already {already}% allocated). The most you can add for these dates is {MAX_LOAD - already}%."
        )


def utilisation_band(percent: int) -> str:
    """Label used on screen and in the dashboard chart."""
    if percent <= 0:
        return "bench"
    if percent < 50:
        return "light"
    if percent < 90:
        return "healthy"
    return "full"
```

backend/app/services/match_service.py

Matching run, ranking and export.

```python
"""Runs the matching engine for a job and persists/reads results.

The engine itself (engine/scoring.py) knows nothing about the database. This
service loads rows, converts them into engine inputs, and saves every result
for a run in ONE transaction (rolled back entirely on any error).
```

```python
"""
from __future__ import annotations

import csv
import io
import logging
import time

from sqlalchemy import func, select
from sqlalchemy.orm import Session, selectinload

from ..engine.embeddings import embedder
from ..engine.scoring import (
    CandidateProfile,
    JobCriteria,
    Weights,
    calculation_string,
    score_candidates,
)
from ..errors import AppError
from ..models import Candidate, CandidateSkill, MatchResult, MatchRun, utcnow
from ..schemas import ResultRow, ResultsOut, RunInfo
from .job_service import get_job, job_out

log = logging.getLogger("hiretrack.match")


def _profiles(db: Session) -> list[CandidateProfile]:
    cands = db.scalars(
        select(Candidate).options(selectinload(Candidate.skills).joinedload(CandidateSkill.skill))
    ).all()
    return [
        CandidateProfile(
            candidate_id=c.candidate_id,
            skills=[cs.skill.name for cs in c.skills],
            years_experience=c.years_experience,
            level=c.level_id,
            resume_summary=c.resume_summary,
            full_name=c.full_name,  # tie-break only
            skills_as_entered={cs.skill.name: (cs.entered_as or cs.skill.name) for cs in c.skills},
        )
        for c in cands
    ]


def run_match(db: Session, job_id: int) -> ResultsOut:
    job = get_job(db, job_id)
    weights = Weights(job.w_skills, job.w_experience, job.w_education, job.w_semantic)
    criteria = JobCriteria(
        required=[s.skill.name for s in job.skills if s.skill_type == "required"],
        preferred=[s.skill.name for s in job.skills if s.skill_type == "preferred"],
        min_experience=job.min_experience,
        min_level=job.min_level_id,
        description=job.job_description,
        weights=weights,
    )
    started = time.perf_counter()
    profiles = _profiles(db)
    if not profiles:
        raise AppError("There are no candidates to match. Add candidates or load the demo data first.")
    scores = score_candidates(criteria, profiles, embedder.encode)
    duration_ms = int((time.perf_counter() - started) * 1000)
    try:
        embedder.save()  # keep newly encoded CVs for next time (no-op if nothing new)
    except Exception:  # pragma: no cover
        log.warning("Could not save embedding cache")

    run_at = utcnow()
    try:
        db.add(
            MatchRun(
                job_id=job_id,
                run_at=run_at,
                duration_ms=duration_ms,
                candidate_count=len(scores),
                w_skills=weights.skills,
                w_experience=weights.experience,
                w_education=weights.education,
                w_semantic=weights.semantic,
            )
        )
        db.add_all(
            [
                MatchResult(
                    candidate_id=s.candidate_id,
                    job_id=job_id,
                    skills_score=s.skills,
                    experience_score=s.experience,
                    education_score=s.education,
                    semantic_score=s.semantic,
                    overall_score=s.overall,
                    rank=s.rank,
                    matched_skills={
                        "required": s.skill_match.matched_required,
                        "preferred": s.skill_match.matched_preferred,
                        "aliases": s.alias_hits,
                    },
                    missing_skills={
                        "required": s.skill_match.missing_required,
                        "preferred": s.skill_match.missing_preferred,
                    },
                    explanation=s.explanation,
                    insights=s.keyword_insights,
                    run_at=run_at,
                )
                for s in scores
```

```
            ]
        )
        db.commit()
    except Exception:
        db.rollback()
        log.exception("Saving match results failed for job id=%s; rolled back", job_id)
        raise
    log.info("Match run job id=%s candidates=%d duration_ms=%d", job_id, len(scores), duration_ms)
    return get_results(db, job_id)


def latest_run(db: Session, job_id: int) -> MatchRun | None:
    return db.scalar(select(MatchRun).where(MatchRun.job_id == job_id).order_by(MatchRun.run_at.desc()).limit(1))


def latest_scores(db: Session, job_id: int) -> dict[int, float]:
    """{candidate_id: overall score} from the latest run for a job (empty if never run)."""
    run = latest_run(db, job_id)
    if run is None:
        return {}
    rows = db.execute(
        select(MatchResult.candidate_id, MatchResult.overall_score).where(
            MatchResult.job_id == job_id, MatchResult.run_at == run.run_at
        )
    ).all()
    return {cid: score for cid, score in rows}


def get_results(db: Session, job_id: int) -> ResultsOut:
    job = get_job(db, job_id)
    run = latest_run(db, job_id)
    if run is None:
        return ResultsOut(job=job_out(db, job), run=None, results=[])
    weights = Weights(run.w_skills, run.w_experience, run.w_education, run.w_semantic)
    rows = db.scalars(
        select(MatchResult)
        .where(MatchResult.job_id == job_id, MatchResult.run_at == run.run_at)
        .order_by(MatchResult.rank)
    ).all()
    from ..models import Application

    stages = dict(
        db.execute(select(Application.candidate_id, Application.stage).where(Application.job_id == job_id)).all()
    )
    out: list[ResultRow] = []
    for r in rows:
        c = r.candidate
        m, x = r.matched_skills or {}, r.missing_skills or {}
        out.append(
            ResultRow(
                result_id=r.result_id,
                rank=r.rank,
                candidate_id=r.candidate_id,
                full_name=c.full_name,
                previous_titles=c.previous_titles,
                years_experience=c.years_experience,
                level_name=c.level.level_name,
                location=c.location,
                pipeline_stage=stages.get(r.candidate_id),
                skills_score=r.skills_score,
                experience_score=r.experience_score,
                education_score=r.education_score,
                semantic_score=r.semantic_score,
                overall_score=r.overall_score,
                matched={"required": m.get("required", []), "preferred": m.get("preferred", [])},
                missing={"required": x.get("required", []), "preferred": x.get("preferred", [])},
                aliases=m.get("aliases", {}),
                explanation=r.explanation or "",
                insights=r.insights or [],
                calculation=_calc(r, weights),
            )
        )
    return ResultsOut(
        job=job_out(db, job),
        run=RunInfo(
            run_at=run.run_at,
            duration_ms=run.duration_ms,
            candidate_count=run.candidate_count,
            w_skills=run.w_skills,
            w_experience=run.w_experience,
            w_education=run.w_education,
            w_semantic=run.w_semantic,
        ),
        results=out,
    )


def _calc(r: MatchResult, w: Weights) -> str:
    from types import SimpleNamespace

    ns = SimpleNamespace(
        skills=r.skills_score,
        experience=r.experience_score,
        education=r.education_score,
        semantic=r.semantic_score,
        overall=r.overall_score,
    )
    return calculation_string(ns, w)


def _csv_safe(value) -> str:
    """Prevent spreadsheet formula injection in exported cells."""
    text = "" if value is None else str(value)
    return "'" + text if text[:1] in ("=", "+", "-", "@", "\t", "\r") else text


```

```python
def results_csv(db: Session, job_id: int) -> tuple[str, str]:
    res = get_results(db, job_id)
    if res.run is None:
        raise AppError("This job has not been matched yet. Run a match first.", status_code=404)
    buf = io.StringIO()
    w = csv.writer(buf)
    w.writerow(
        [
            "Rank", "Candidate", "Overall", "Skills", "Experience", "Education", "Semantic",
            "Matched required", "Missing required", "Matched preferred", "Missing preferred",
            "Explanation", "Calculation",
        ]
    )
    for r in res.results:
        w.writerow(
            [_csv_safe(v) for v in (
                r.rank, r.full_name, r.overall_score, r.skills_score, r.experience_score,
                r.education_score, r.semantic_score,
                "; ".join(r.matched.required), "; ".join(r.missing.required),
                "; ".join(r.matched.preferred), "; ".join(r.missing.preferred),
                r.explanation, r.calculation,
            )]
        )
    slug = "".join(ch if ch.isalnum() else "-" for ch in res.job.job_title.lower()).strip("-")
    fname = f"hiretrack-{slug}-{res.run.run_at:%Y%m%d-%H%M}.csv"
    return buf.getvalue(), fname


def stats(db: Session) -> dict:
    from ..models import JobSpecification

    n_cand = db.scalar(select(func.count(Candidate.candidate_id))) or 0
    n_jobs = db.scalar(select(func.count(JobSpecification.job_id))) or 0
    last = db.scalar(select(MatchRun).order_by(MatchRun.run_at.desc()).limit(1))
    avg = None
    title = None
    if last is not None:
        avg = db.scalar(
            select(func.avg(MatchResult.overall_score)).where(
                MatchResult.job_id == last.job_id, MatchResult.run_at == last.run_at
            )
        )
        title = db.scalar(select(JobSpecification.job_title).where(JobSpecification.job_id == last.job_id))
    return {
        "candidates": n_cand,
        "jobs": n_jobs,
        "last_run_at": last.run_at if last else None,
        "last_run_job_title": title,
        "last_run_job_id": last.job_id if last else None,
        "average_score": round(avg, 1) if avg is not None else None,
        "embedder": embedder.backend_name,
    }
```

backend/app/services/pipeline_service.py

Applications, stage changes and hiring.

```python
"""Applicant pipeline: applications, stage changes and the audit trail.

Every stage change writes an ApplicationEvent row in the same transaction as
the change itself, so the history can never disagree with the current stage.
Moving an application to "hired" also adds the person to the staff pool, so a
Resource Manager can allocate them to projects straight away.
"""
from __future__ import annotations

import logging

from sqlalchemy import func, or_, select
from sqlalchemy.orm import Session, selectinload

from ..engine import pipeline as rules
from ..errors import AppError, Conflict, NotFound
from ..models import Application, ApplicationEvent, Candidate, JobSpecification, StaffMember, User, utcnow
from ..schemas import ApplicationEventOut, ApplicationIn, ApplicationOut, BulkApplicationIn, StageChangeIn

log = logging.getLogger("hiretrack.pipeline")


def _rejected_from(app: Application) -> str | None:
    """The stage this application was at when it was rejected (for reopening)."""
    for ev in reversed(app.events):
        if ev.to_stage == rules.REJECTED:
            return ev.from_stage
    return None


def to_out(app: Application, with_events: bool = False) -> ApplicationOut:
    c = app.candidate
    events = None
    if with_events:
        events = [
            ApplicationEventOut(
                event_id=e.event_id,
                from_stage=e.from_stage,
                to_stage=e.to_stage,
                changed_by_name=e.user.full_name if e.user else None,
                changed_at=e.changed_at,
                note=e.note,
            )
            for e in app.events
```

```
        ]
    return ApplicationOut(
        application_id=app.application_id,
        candidate_id=app.candidate_id,
        candidate_name=c.full_name,
        candidate_titles=c.previous_titles,
        candidate_location=c.location,
        years_experience=c.years_experience,
        job_id=app.job_id,
        job_title=app.job.job_title,
        stage=app.stage,
        match_score=app.match_score,
        applied_at=app.applied_at,
        updated_at=app.updated_at,
        allowed_moves=rules.allowed_moves(app.stage, _rejected_from(app)),
        events=events,
    )


def _query():
    return select(Application).options(selectinload(Application.events).joinedload(ApplicationEvent.user))


def list_applications(
    db: Session, job_id: int | None = None, stage: str | None = None, search: str | None = None
) -> list[ApplicationOut]:
    q = _query().join(Application.candidate)
    if job_id is not None:
        q = q.where(Application.job_id == job_id)
    if stage:
        q = q.where(Application.stage == stage)
    if search and search.strip():
        term = f"%{search.strip().lower()}%"
        q = q.where(
            or_(
                func.lower(Candidate.full_name).like(term),
                func.lower(func.coalesce(Candidate.previous_titles, "")).like(term),
            )
        )
    q = q.order_by(Application.updated_at.desc(), Application.application_id.desc())
    return [to_out(a) for a in db.scalars(q).unique().all()]


def get_application(db: Session, application_id: int) -> Application:
    app = db.scalar(_query().where(Application.application_id == application_id))
    if app is None:
        raise NotFound(f"Application {application_id} not found")
    return app


def _check_refs(db: Session, candidate_id: int, job_id: int) -> JobSpecification:
    if db.get(Candidate, candidate_id) is None:
        raise NotFound(f"Candidate {candidate_id} not found", field="candidate_id")
    job = db.get(JobSpecification, job_id)
    if job is None:
        raise NotFound(f"Job {job_id} not found", field="job_id")
    if job.status != "open":
        raise AppError("This job is closed and is not accepting applicants", field="job_id", status_code=422)
    return job


def create_application(db: Session, data: ApplicationIn, user: User) -> Application:
    _check_refs(db, data.candidate_id, data.job_id)
    exists = db.scalar(
        select(Application.application_id).where(
            Application.candidate_id == data.candidate_id, Application.job_id == data.job_id
        )
    )
    if exists:
        raise Conflict("This candidate is already in the pipeline for this job", field="candidate_id")
    now = utcnow()
    app = Application(
        candidate_id=data.candidate_id,
        job_id=data.job_id,
        stage="applied",
        match_score=data.match_score,
        applied_at=now,
        updated_at=now,
    )
    app.events.append(
        ApplicationEvent(from_stage=None, to_stage="applied", changed_by=user.user_id, changed_at=now, note=data.note)
    )
    db.add(app)
    db.commit()
    log.info("Application id=%s created (candidate=%s job=%s)", app.application_id, data.candidate_id, data.job_id)
    return get_application(db, app.application_id)


def bulk_add(db: Session, data: BulkApplicationIn, user: User) -> dict:
    """Add several candidates (e.g. a shortlist from Results) to one job's pipeline.

    Candidates already in the pipeline for this job are skipped, not errors.
    The match score from the latest run is stored with each new application.
    """
    from .match_service import latest_scores

    job = db.get(JobSpecification, data.job_id)
    if job is None:
        raise NotFound(f"Job {data.job_id} not found", field="job_id")
    if job.status != "open":
        raise AppError("This job is closed and is not accepting applicants", field="job_id", status_code=422)
    ids = list(dict.fromkeys(data.candidate_ids))
    known = set(db.scalars(select(Candidate.candidate_id).where(Candidate.candidate_id.in_(ids))))
    missing = [i for i in ids if i not in known]
    if missing:
        raise NotFound(f"Candidate {missing[0]} not found", field="candidate_ids")
```

```
    already = set(
        db.scalars(
            select(Application.candidate_id).where(Application.job_id == job.job_id, Application.candidate_id.in_(ids))
        )
    )
    scores = latest_scores(db, job.job_id)
    now = utcnow()
    added = 0
    for cid in ids:
        if cid in already:
            continue
        app = Application(
            candidate_id=cid, job_id=job.job_id, stage="applied", match_score=scores.get(cid), applied_at=now,
updated_at=now
        )
        app.events.append(
            ApplicationEvent(
                from_stage=None, to_stage="applied", changed_by=user.user_id, changed_at=now, note="Added from match
results"
            )
        )
        db.add(app)
        added += 1
    db.commit()
    log.info("Bulk-added %d applications to job id=%s", added, job.job_id)
    return {"added": added, "skipped": len(ids) - added}


def _ensure_staff(db: Session, app: Application) -> StaffMember:
    """Create a staff record for a hired candidate (once)."""
    staff = db.scalar(select(StaffMember).where(StaffMember.candidate_id == app.candidate_id))
    if staff is None:
        c = app.candidate
        staff = StaffMember(
            full_name=c.full_name,
            email=c.email,
            role_title=app.job.job_title,
            location=c.location or app.job.location,
            candidate_id=c.candidate_id,
            source="hired",
        )
        db.add(staff)
    else:
        staff.is_active = True
        staff.role_title = app.job.job_title
    return staff


def change_stage(db: Session, application_id: int, data: StageChangeIn, user: User) -> Application:
    app = get_application(db, application_id)
    try:
        rules.check_move(app.stage, data.stage, _rejected_from(app))
    except rules.StageError as exc:
        raise AppError(str(exc), field="stage", status_code=422) from exc
    if data.stage == "hired" and app.job.status != "open":
        raise AppError("This job is closed, so nobody else can be hired for it", field="stage", status_code=422)
    now = utcnow()
    app.events.append(
        ApplicationEvent(
            from_stage=app.stage, to_stage=data.stage, changed_by=user.user_id, changed_at=now, note=data.note or None
        )
    )
    app.stage = data.stage
    app.updated_at = now
    if data.stage == "hired":
        _ensure_staff(db, app)
    db.commit()
    log.info("Application id=%s moved to %s by user id=%s", application_id, data.stage, user.user_id)
    db.expire_all()
    return get_application(db, application_id)


def delete_application(db: Session, application_id: int) -> None:
    app = get_application(db, application_id)
    if app.stage == "hired":
        raise AppError("A hired application cannot be deleted; it is part of the staff record", status_code=422)
    db.delete(app)
    db.commit()
    log.info("Deleted application id=%s", application_id)


def stage_counts(db: Session, job_id: int | None = None) -> dict[str, int]:
    q = select(Application.stage, func.count(Application.application_id)).group_by(Application.stage)
    if job_id is not None:
        q = q.where(Application.job_id == job_id)
    return {stage: n for stage, n in db.execute(q).all()}


def funnel(db: Session) -> list[dict]:
    current = stage_counts(db)
    # For rejected applications, find the stage they were rejected from.
    rows = db.execute(
        select(ApplicationEvent.application_id, ApplicationEvent.from_stage)
        .join(Application, Application.application_id == ApplicationEvent.application_id)
        .where(Application.stage == rules.REJECTED, ApplicationEvent.to_stage == rules.REJECTED)
        .order_by(ApplicationEvent.event_id)
    ).all()
    rejected_from = {app_id: frm for app_id, frm in rows}  # last rejection wins
    furthest: dict[str, int] = {}
    for stage, n in current.items():
        if stage != rules.REJECTED:
            furthest[stage] = furthest.get(stage, 0) + n
    for frm in rejected_from.values():
        s = rules.furthest_stage(rules.REJECTED, frm)
        furthest[s] = furthest.get(s, 0) + 1
    return rules.funnel(furthest, current)
```

backend/app/services/resource_service.py

Staff, projects and allocations.

```python
"""Resource allocation: staff pool, projects, allocations and utilisation.

Utilisation is worked out for a reference day (today by default) from the
allocations that cover that day on projects that are planned or active.
Allocations on completed or on-hold projects do not count towards anyone's load.
"""
from __future__ import annotations

import logging
from datetime import date

from sqlalchemy import func, select
from sqlalchemy.orm import Session, selectinload

from ..engine.allocation import Booking, check_new_booking, load_on, utilisation_band
from ..errors import AppError, Conflict, NotFound
from ..models import Allocation, Project, StaffMember
from ..schemas import AllocationIn, AllocationOut, ProjectIn, ProjectOut, StaffIn, StaffOut

log = logging.getLogger("hiretrack.resources")

COUNTING = ("planned", "active")  # project statuses that use up a person's time


def today() -> date:
    return date.today()


# ---------------------------------------------------------------------- staff
def _bookings(allocs: list[Allocation]) -> list[Booking]:
    return [
        Booking(a.allocation_percent, a.start_date, a.end_date, a.allocation_id)
        for a in allocs
        if a.project.status in COUNTING
    ]


def staff_to_out(s: StaffMember, on: date | None = None) -> StaffOut:
    on = on or today()
    load = load_on(_bookings(s.allocations), on)
    current = sorted(
        {a.project.name for a in s.allocations if a.start_date <= on <= a.end_date and a.project.status in COUNTING}
    )
    return StaffOut(
        staff_id=s.staff_id,
        full_name=s.full_name,
        email=s.email,
        role_title=s.role_title,
        location=s.location,
        source=s.source,
        candidate_id=s.candidate_id,
        is_active=s.is_active,
        utilisation=load,
        band=utilisation_band(load) if s.is_active else "inactive",
        current_projects=current,
    )


def _staff_query():
    return select(StaffMember).options(selectinload(StaffMember.allocations).joinedload(Allocation.project))


def list_staff(db: Session, search: str | None = None, band: str | None = None, on: date | None = None) -> list[StaffOut]:
    q = _staff_query().order_by(func.lower(StaffMember.full_name))
    if search and search.strip():
        term = f"%{search.strip().lower()}%"
        q = q.where(func.lower(StaffMember.full_name).like(term) | func.lower(StaffMember.role_title).like(term))
    rows = [staff_to_out(s, on) for s in db.scalars(q).unique().all()]
    if band:
        rows = [r for r in rows if r.band == band]
    return rows


def get_staff(db: Session, staff_id: int) -> StaffMember:
    s = db.scalar(_staff_query().where(StaffMember.staff_id == staff_id))
    if s is None:
        raise NotFound(f"Staff member {staff_id} not found")
    return s


def create_staff(db: Session, data: StaffIn) -> StaffMember:
    s = StaffMember(
        full_name=data.full_name,
        email=data.email,
        role_title=data.role_title,
        location=data.location or None,
        is_active=data.is_active,
        source="existing",
    )
    db.add(s)
    db.commit()
    log.info("Created staff id=%s", s.staff_id)
    return get_staff(db, s.staff_id)


def update_staff(db: Session, staff_id: int, data: StaffIn) -> StaffMember:
```

```python
    s = get_staff(db, staff_id)
    s.full_name = data.full_name
    s.email = data.email
    s.role_title = data.role_title
    s.location = data.location or None
    s.is_active = data.is_active
    db.commit()
    db.expire_all()
    return get_staff(db, staff_id)


def delete_staff(db: Session, staff_id: int) -> None:
    s = get_staff(db, staff_id)
    db.delete(s)
    db.commit()
    log.info("Deleted staff id=%s", staff_id)


# ---------------------------------------------------------------- projects
def alloc_to_out(a: Allocation, on: date | None = None) -> AllocationOut:
    on = on or today()
    return AllocationOut(
        allocation_id=a.allocation_id,
        staff_id=a.staff_id,
        staff_name=a.staff.full_name,
        staff_role=a.staff.role_title,
        project_id=a.project_id,
        project_name=a.project.name,
        role_on_project=a.role_on_project,
        allocation_percent=a.allocation_percent,
        start_date=a.start_date,
        end_date=a.end_date,
        is_current=a.start_date <= on <= a.end_date,
    )


def project_to_out(p: Project, with_allocations: bool = False) -> ProjectOut:
    on = today()
    current = [a for a in p.allocations if a.start_date <= on <= a.end_date]
    return ProjectOut(
        project_id=p.project_id,
        name=p.name,
        client=p.client,
        description=p.description,
        start_date=p.start_date,
        end_date=p.end_date,
        status=p.status,
        team_size=len({a.staff_id for a in p.allocations}),
        allocated_percent=sum(a.allocation_percent for a in current),
        allocations=(
            [alloc_to_out(a, on) for a in sorted(p.allocations, key=lambda a: (a.start_date, a.staff.full_name))]
            if with_allocations
            else None
        ),
    )


def _project_query():
    return select(Project).options(selectinload(Project.allocations).joinedload(Allocation.staff))


def list_projects(db: Session, status: str | None = None) -> list[ProjectOut]:
    q = _project_query().order_by(Project.start_date.desc(), Project.name)
    if status:
        q = q.where(Project.status == status)
    return [project_to_out(p) for p in db.scalars(q).unique().all()]


def get_project(db: Session, project_id: int) -> Project:
    p = db.scalar(_project_query().where(Project.project_id == project_id))
    if p is None:
        raise NotFound(f"Project {project_id} not found")
    return p


def _name_taken(db: Session, name: str, except_id: int | None = None) -> bool:
    q = select(Project.project_id).where(func.lower(Project.name) == name.lower())
    if except_id is not None:
        q = q.where(Project.project_id != except_id)
    return db.scalar(q) is not None


def create_project(db: Session, data: ProjectIn) -> Project:
    if _name_taken(db, data.name):
        raise Conflict("A project with this name already exists", field="name")
    p = Project(**data.model_dump())
    db.add(p)
    db.commit()
    log.info("Created project id=%s", p.project_id)
    return get_project(db, p.project_id)


def update_project(db: Session, project_id: int, data: ProjectIn) -> Project:
    p = get_project(db, project_id)
    if _name_taken(db, data.name, except_id=project_id):
        raise Conflict("A project with this name already exists", field="name")
    outside = [a for a in p.allocations if a.start_date < data.start_date or a.end_date > data.end_date]
    if outside:
        raise AppError(
            f"{len(outside)} allocation(s) fall outside the new dates. Change or remove them first.",
            field="start_date",
            status_code=422,
        )
    for k, v in data.model_dump().items():
        setattr(p, k, v)
    db.commit()
```

```
    db.expire_all()
    return get_project(db, project_id)


def delete_project(db: Session, project_id: int) -> None:
    p = get_project(db, project_id)
    db.delete(p)
    db.commit()
    log.info("Deleted project id=%s", project_id)


# ------------------------------------------------------------ allocations
def _validate_allocation(db: Session, data: AllocationIn, ignore_id: int | None = None) -> tuple[StaffMember, Project]:
    staff = get_staff(db, data.staff_id)
    project = get_project(db, data.project_id)
    if not staff.is_active:
        raise AppError("This staff member is inactive and cannot be allocated", field="staff_id", status_code=422)
    if project.status == "completed":
        raise AppError("This project is completed; allocations cannot be added", field="project_id", status_code=422)
    if data.start_date < project.start_date or data.end_date > project.end_date:
        raise AppError(
            f"Dates must fall within the project ({project.start_date:%d %b %Y} to {project.end_date:%d %b %Y})",
            field="start_date",
            status_code=422,
        )
    if project.status in COUNTING:
        try:
            check_new_booking(
                _bookings(staff.allocations),
                Booking(data.allocation_percent, data.start_date, data.end_date),
                ignore_id=ignore_id,
            )
        except ValueError as exc:
            raise AppError(str(exc), field="allocation_percent", status_code=422) from exc
    return staff, project


def get_allocation(db: Session, allocation_id: int) -> Allocation:
    a = db.get(Allocation, allocation_id)
    if a is None:
        raise NotFound(f"Allocation {allocation_id} not found")
    return a


def create_allocation(db: Session, data: AllocationIn) -> Allocation:
    _validate_allocation(db, data)
    a = Allocation(**data.model_dump())
    db.add(a)
    db.commit()
    log.info("Allocated staff id=%s to project id=%s at %s%%", data.staff_id, data.project_id, data.allocation_percent)
    db.expire_all()
    return get_allocation(db, a.allocation_id)


def update_allocation(db: Session, allocation_id: int, data: AllocationIn) -> Allocation:
    a = get_allocation(db, allocation_id)
    _validate_allocation(db, data, ignore_id=allocation_id)
    for k, v in data.model_dump().items():
        setattr(a, k, v)
    db.commit()
    db.expire_all()
    return get_allocation(db, allocation_id)


def delete_allocation(db: Session, allocation_id: int) -> None:
    a = get_allocation(db, allocation_id)
    db.delete(a)
    db.commit()
    log.info("Deleted allocation id=%s", allocation_id)


def list_allocations(db: Session, staff_id: int | None = None, project_id: int | None = None) -> list[AllocationOut]:
    q = select(Allocation).order_by(Allocation.start_date.desc())
    if staff_id is not None:
        q = q.where(Allocation.staff_id == staff_id)
    if project_id is not None:
        q = q.where(Allocation.project_id == project_id)
    return [alloc_to_out(a) for a in db.scalars(q).unique().all()]


# ------------------------------------------------------------ summary
def utilisation_summary(db: Session, on: date | None = None) -> dict:
    staff = [s for s in list_staff(db, on=on) if s.is_active]
    bands = {"bench": 0, "light": 0, "healthy": 0, "full": 0}
    for s in staff:
        bands[s.band] += 1
    avg = round(sum(s.utilisation for s in staff) / len(staff), 1) if staff else None
    return {"active_staff": len(staff), "average_utilisation": avg, "bands": bands}
```